

EMU 3.3 User's Guide



13-May-2024
Carl Timmer

© Thomas Jefferson National Accelerator Facility
12000 Jefferson Ave
Newport News, VA 23606
Phone 757.269.7100

Table of Contents

1	Introduction	4
1.1.	<i>Input / Output.....</i>	5
1.2.	<i>Modules.....</i>	5
1.3.	<i>Configuring.....</i>	6
1.4.	<i>Monitoring, and Controlling.....</i>	6
2	Getting, Building, and Installing the EMU	7
2.1.	<i>Getting the EMU.....</i>	7
2.2.	<i>Compiling Java.....</i>	7
2.3.	<i>Building Documentation.....</i>	9
3	Data Input and Output	10
3.1.	<i>Transports.....</i>	10
3.1.1.	<i>FIFOs.....</i>	10
3.1.2.	<i>Files</i>	11
3.1.3.	<i>cMsg messages in cMsg domain.....</i>	11
3.1.4.	<i>Sockets in cMsg emu domain.....</i>	11
3.1.5.	<i>ET</i>	12
3.1.6.	<i>Streaming TCP Socket</i>	14
3.1.7.	<i>Streaming UDP Socket.....</i>	15
3.2.	<i>Channels</i>	15
3.2.1.	<i>FIFOs.....</i>	15
3.2.2.	<i>Files</i>	16
3.2.3.	<i>cMsg – cMsg domain.....</i>	18
3.2.4.	<i>cMsg – emu domain</i>	19
3.2.5.	<i>ET</i>	21
3.2.6.	<i>Streaming TCP Socket</i>	23
3.2.7.	<i>Streaming UDP Socket.....</i>	24
3.2.8.	<i>Event Recorder – special rules.....</i>	26
4	Modules	27
4.1.	<i>Config File</i>	27
4.2.	<i>Fast Event Builder</i>	28
4.3.	<i>Stream Aggregator.....</i>	30
4.4.	<i>Event Recording.....</i>	31
4.5.	<i>ROC Simulation</i>	32
4.6.	<i>Trigger Supervisor Simulation.....</i>	33
4.7.	<i>Farm Controller.....</i>	34
5	Running an EMU with Run Control	35
5.1.	<i>Config File Final Form.....</i>	35

5.2.	<i>Creating EMUs</i>	36
5.3.	<i>Platform connection</i>	37
5.4.	<i>Running a single EMU</i>	37
5.5.	<i>Running Multiple EMUs</i>	37
6	Running an EMU with the Debug GUI	38
7	Developer's Details	41
7.1.	<i>cMsg Run Control Connection</i>	41
7.2.	<i>Data Flow</i>	42
7.3.	<i>Triggered Data Format Channels</i>	42
7.3.1.	ET Channels	42
7.3.2.	cMsg Channels, cMsg domain	44
7.3.3.	cMsg Channels, emu domain	45
7.4.	<i>Streaming Data Format Channels</i>	46
7.4.1.	UDP	46
7.4.1.	TCP.....	46
7.5.	<i>Simulated ROCs and TS</i>	46
7.5.1.	Trigger Supervisor.....	47
7.5.2.	ROCs	47
7.6.	<i>Simulated Fixed-Rate ROCs and TS</i>	48
7.7.	<i>Evio Events Per ET Buffer</i>	49
8	Fast Ring Buffers	51
8.1.	<i>Locks are Bad</i>	51
8.2.	<i>Cache Lines</i>	52
8.3.	<i>The Trouble with Queues</i>	52
8.4.	<i>Disruptor Design</i>	53
8.5.	<i>Disruptor Use in a Previous EB</i>	53
8.6.	<i>Disruptor Use in the Byte Buffer Supply</i>	55
8.7.	<i>General Disruptor Use in the Emu</i>	56
8.8.	<i>Ring Buffer Example Code</i>	57
A.	CODA Types	59

Chapter 1

1 Introduction

Prior to CODA (CEBAF online data acquisition) version 3, CODA's data-handling software components were self-contained, independent software entities. These components included the Readout Controller (ROC) which ran on embedded computers using the realtime operating system vxWorks. Its task was to read the data-producing hardware modules, package the data and send it to the next component. Next in line was the Event Builder (EB) which took the data from all of the ROCs and made one EVIO event out of it. Finally, there was the Event Recorder (ER) which took the nicely packaged data and wrote it to a file.

Each component's communications had to be carefully coordinated with the other components and each was also individually responsible to communicate with run control and respond to its commands. As you can imagine, much of the code was redundant between the ROC, EB, and ER.

With the development of the ET system, which was used in CODA version 2 to transport data from the EB to users and to the ER, it was a small jump to use it between the ROC and the EB as well. The additional availability of the cMsg message-passing software package made it another tiny hop to replace all run control communication code with calls to cMsg. Between these 2 pieces of software, all the interprocess communication needs were met and all the data transfer software was abstracted out of the CODA components.

While CODA version 3 is built on its ability to use ET and cMsg to do all the "talking", another area of abstracting functionality involves the EB and ER. Both are very similar in functionality in that they both read data, do something to the data, write the data, and respond to run control commands. Right away it's obvious that the reading in, writing out, and run control parts can be identical between the EB and ER. It's also a fairly simple matter to take the middle part (doing something with the data) and make that a plug-in. This is the fundamental structure of the EMU. It's a framework to ease development by taking out all the identical CODA component pieces and programming them once for all. It allows selection of standard inputs and outputs and it accepts run control commands. All the user must do is write the plug-in to handle the data and respond to the incoming commands.

1.1. *Input / Output*

The EMU is designed to read and write evio format data. It may accept such data by 4 different means or **transports**:

- through the ET system
- in cMsg messages using the cMsg domain pub/sub server
- in cMsg messages using the cMsg emu domain TCP sockets, and
- from files.

A single, different transport deals with each of these four types of data transfer. Multiple transports can be used in a single EMU. Each of these transports can have multiple **channels** in a single EMU as well. Each channel is a single connection to an ET system or cMsg server, or opening of a file.

The transport handling code is abstracted from the rest of the EMU in such a way as to be able to add other means of I/O. To document the complete interface to accomplish this is too much work for a task best left to the DAQ group itself. Thus, if a user needs to interface with another type of I/O, contact the DAQ group to make arrangements for adding the necessary code.

The EMU expects ET events or buffers to contain evio data - simple enough. In cMsg, it expects the byte array to contain the evio data (in Java, `msg.getBytes()`) - equally simple. Files are self-explanatory.

1.2. *Modules*

Between the input and output channels are **modules**. These are Java classes that take evio events from an input channel and place evio events on an output channel. There can be more than one module to a single EMU. The modules are ordered sequentially so that the data flows in one path through the EMU. Fifos are I/O channels used between modules.

The data flow looks like Figure 1 below. An input channel receives data through an ET event, cMsg message or file. It parses the incoming data into evio events. Each evio event received is placed into its queue. That queue along with all the other input channel queues are inputs to the first module. That module takes each evio event, processes it, and then places it in an output channel or fifo. At this point either the next module gets, processes, and passes it on, or it goes to a queue of one of the output channels. The channel then gets, processes, and sends it somewhere else.

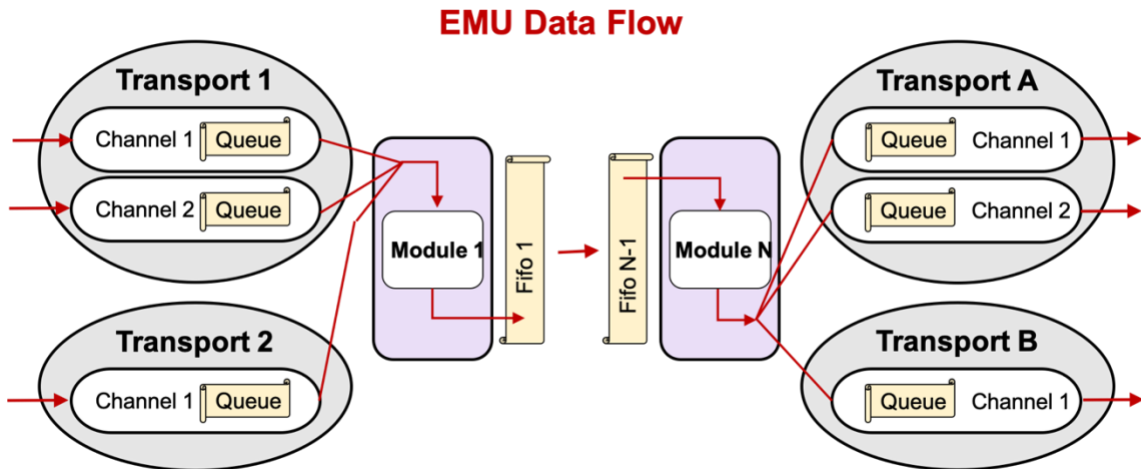


Figure 1.1 EMU Data Flow

1.3. *Configuring*

Each EMU can be configured through an XML configuration file with specific elements and attributes (details are contained in later chapters). They can be complicated to configure since there are such a large number of parameters one can tweak. A large part of the complication is due to interprocess communication.

1.4. *Monitoring, and Controlling*

The EMU can be visually monitored and controlled by means of a built-in debugging GUI. This GUI is useful when, for example, the user is testing a module and no run control platform is operating, since it can send the run control commands to the EMU.

Chapter 2

2 Getting, Building, and Installing the EMU

You must install Java version 8 or higher since all pre-built CODA jar files are compiled with it.

2.1. *Getting the EMU*

The EMU package documentation can be found on the JLab Data Acquisition Group CODA wiki at <http://coda.jlab.org>. However, all this site does is direct one to the github repository in which the package is stored. For Java users, a pre-built jar file is already available on the CODA site and is also contained in the code downloaded from github and usually is all that is needed.

To install all of the EMU do:

```
git clone -b emu-3.3 https://github.com/JeffersonLab/emu.git
```

This will give you a full EMU distribution with the top-level directory being **emu**. It will be on the emu-3.3 branch which, strangely enough, corresponds to EMU version 3.3. The documentation is available on the above-mentioned web site but also exists in the **doc** subdirectory of the full distribution.

2.2. *Compiling Java*

One can find the pre-built emu-3.3.jar file in the repository in the **java/jars/java8** directory built with Java 8, or one can also find that jar in the **java/jars/java15** directory built with Java 15, or it can be generated. In either case, put the jar file into your classpath and run your java application.

When generating the jar file, it's advisable to use java version 8 or higher since all other pre-built CODA jar files have been compiled with java 8. Ant must be installed on your system (<http://ant.apache.org>). Simply execute:

```
ant jar
```

in the top-level directory. To get a list of options with ant, type:

```
ant help
```

Following is a table of the available options:

ant command	action
ant, ant compile	compile all java source code
ant help	print out usage
ant env	print out value of build file variables
ant clean	remove all class files
ant cleanall	remove all generated files - not including documentation
ant jar	compile and create emu jar file
ant install	create cmsg jar file and install all jars into 'prefix' if given on command line by -Dprefix=dir', else install into CODA if defined (ET jar file is not installed)
ant uninstall	remove all jar files previously installed into 'prefix' if given on command line by -Dprefix=dir', else installed into CODA if defined (ET jar file is not removed)
ant all	do clean, compile, then create emu jar file
ant javadoc	create javadoc documentation for user
ant developdoc	create javadoc documentation for user & developer
ant undoc	remove all javadoc documentation
ant prepare	create necessary directories

The generated jar file is placed in **build/lib**. Included in the **java/jars** subdirectory are all auxiliary jar files used by the emu. When installing the emu jar, the disruptor and swing-layout jars are also installed. The emu also needs the cMsg, et, and evio jars as well, but those will not be installed from this repository. Check those individual package web sites for more information.

2.3. *Building Documentation*

All documentation is available from <http://coda.jlab.org>. However, if using the downloaded distribution from github, some of the documentation needs to be generated and some already exists. For existing docs look in **doc/users_guide** for pdf and Word format documents.

Some of the documentation is in the source code itself and must be generated and placed into its own directory. The java code is documented with javadoc comments. The generated javadoc from that is placed in the **doc/javadoc** directory. To view the html documentation, just point your browser to the index.html file in that directory. This documentation can be generated by typing:

```
ant javadoc
```

for user-level documentation, or

```
ant developdoc
```

for developer-level documentation. To remove it:

```
ant undoc
```

Chapter 3

3 Data Input and Output

EMU's can be complicated to configure since there are such a large number of parameters one can set. Most of the complication is due to interprocess communication. To review, EMUs may use 4 different means of communication to the outside world: ET systems, cMsg messages in cMsg domain, TCP sockets in cMsg emu domain, and files. The data being sent to, through, and out of the EMU is in EVIO version 4 or 6 format. The channel objects in the EMU read and write only in this format.

The input data is read, parsed, and broken up into individual EVIO events (top level banks). These events are what are placed in the internal queues and FIFOs seen in the first chapter's diagram. Thus, modules deal only with evio events directly. Similarly, the output transport objects take the events from their queues and repackage them to be sent in proper EVIO format.

So how does one go about the business of specifying all this data movement? Each EMU is configured through an XML file with 2 major sections. Parts of the I/O config come in both sections. One major section specifies data transport mechanisms while the other specifies the modules. Under each module definition come individual data channels from one of the defined transports. The following will walk the user through creating the I/O portion of these config files.

3.1. *Transports*

One major part of a config file is contained in the single XML element, *transports*, usually in the first lines. There are 4 different types of transports to the outside world that may be used: cMsg, emu, ET, and files. And there is 1 type of transport between modules, FIFOs. Each of these is configured quite differently. The xml element defining a transport uses the name *server*.

3.1.1. FIFOs

The FIFO is a type of transport that is built into the EMU and needs no specification in transports section of the config file.

3.1.2. Files

Specifying the parameters to define a file transport requires only 2 things:

1. the user-given **name**, and
2. the **class** which is always and exactly "File" (case sensitive).

```
<transports>
  <server name="myFile" class="File" />
</transports>
```

3.1.3. cMsg messages in cMsg domain

Specifying the parameters to define a cMsg transport requires 3 things:

1. the user-given **name** which will also be used as the cMsg client name and therefore must be unique to the cMsg server,
2. the cMsg **udl** used to connect to the cMsg server and
3. the **class** which is always and exactly "Cmsg" (case sensitive)

The udl may be set to “**platform**” in which case the udl is internally set to the cMsg server inside the run control platform being used and the namespace is set to “CODA”. See the cMsg documentation to learn more about the format of udl's. In practice, this transport is never used in CODA as the cMsg pub/sub system was designed for the low-rate sending of small messages, not high-speed data acquisition.

```
<transports>
  <server name="mycMsg" udl="cMsg://host:port/cMsg/nspce" class="Cmsg" />
  <server name="yourcMsg" udl="platform" class="Cmsg" />
</transports>
```

3.1.4. Sockets in cMsg emu domain

Specifying the parameters to define a cMsg, emu domain transport when **receiving** data requires 3 things:

1. the user-given **name** which will also be referred to by any channels that use it,
2. the TCP **port** number used for communication, and
3. the **class** which is always and exactly "Emu" (case sensitive).

Note that the xml element name is **client** since it is receiving data even though it acts, in fact, as a TCP server awaiting connections from DAQ components upstream. A unique port is required for each component in the DAQ system that uses this protocol. If a port is not specified, it defaults to 46100.

```
<transports>
  <client name="emuSocket1" port="46101" class="Emu" />
</transports>
```

When **sending** data with this transport, the port does not need to be set (since that is set in the channel) and the xml element name is **server** since it is sending data.

```
<transports>
  <server name="emuSocket1" class="Emu" />
</transports>
```

To go into a little more detail, a downstream component, say an EB, will start such a server in the transport object's download transition which behaves somewhat like an rc multicast server. When it starts up, it starts 2 servers, a UDP and a TCP server. The UDP server then listens for multicasts, but only accepts packets from upstream components in the configuration which must connect to it, say Roc1. (Thus, multiple sessions can coexist with servers of the same port & EXPID if they serve different components). It sends a packet back to the upstream component (say Roc1) with enough information so that it can make a connection to the TCP server. It stays this way until prestart when Roc1's output channel will connect to the TCP server and send data over that socket. See the section below on the cMsg emu domain channel for more info.

It is also possible to bypass the step of multicasting to find the listening TCP server and connect to the TCP server directly by specifying its host and port. In practice, this is what is now done in CODA since it's much more efficient and less time consuming. In the download transition, the upstream component (say Roc1) will receive the necessary host and port from run control to make a direct TCP connection. More on this in the emu channel section.

3.1.5. ET

The EMU uses the ET system in 2 different ways. In the first, the EMU "owns" an ET system. It creates the ET system, uses it, and removes it when finished. The second method is to simply connect to an existing system, use it, then disconnect when finished.

The xml element defining an ET transport uses the name **server**. The following is a table containing each of its xml attributes (case sensitive), whether it's required, its meaning, and its acceptable values:

Attribute	Required	Function	Allowed Value(s)
All ET systems			
name	✓	User given name of transport	Any string.
class	✓	Java class for transport object	The exact string, "Et".
etName		ET system name	Et file name. Defaults to /tmp/<EXPID>_<EMU name>, where EXPID is an environmental variable or given on command line
create		Does EMU create the ET system?	Case indep. "true", "on", or "yes". Anything else is false. Default = false.
port		TCP port.	Integer > 1023 and < 65536. Defaults to 11111.
uPort		UDP port.	Integer > 1023 and < 65536. Defaults to broadcasting port, 11111.
wait		How many seconds to wait for an ET system when connecting	Integer >= 0 seconds. Defaults to 0.
mAddr		Multicast address	Any valid multicast address. Needed only if method = "mcast" or "cast".
When creating ET systems			
type		Create java-based or C-based ET sys.	The case indep. string "java" for java-based, else C-based.
eventNum	✓	Number of events.	Integer > 0.
eventSize	✓	Size of event data in bytes.	Integer > 0.
groups		Number of groups to divide events into.	Integer > 0 and <= eventNum . Defaults to 1.
revBuf		TCP receive buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
sendBuf		TCP send buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
noDelay		TCP NODELAY value	Case indep. "true", "on", or "yes". Anything else is false. Default = false.
When connecting to existing ET systems			
method		Method to connect with.	Case indep "direct" for direct to server, "mcast" for multicasting, "bcast" for broadcasting, "cast" for both.
host		Host running ET system.	Case indep "anywhere", "remote", "local" denotes where on network to look for ET. Or name of host. Defaults to "anywhere".
bAddr		Broadcast address.	Any valid broadcast address. Needed only if method = "bcast" or "cast".
subnet		Preferred IP address / subnet to use for ET communication if possible.	May be preferred IP address of local host which is used to for ET I/O. May also be broadcast address of subnet preferred for communication which gets changed to host IP address on that subnet if possible.

Now for a few examples. The following is an example of a config file entry that can be used to have the EMU create and control an ET system:

```
<transports>
  <server name="myEt" class="Et" etName="/tmp/Eb1" create="true"
    port="54321" uPort="12345" eventNum="1000" eventSize="2100000"
    groups="2" wait="10" recvBuf="3000000" sendBuf="130000"/>
</transports>
```

Notice that **method** and **host** are not defined. The EMU will automatically attach to created ET systems using a direct connection to a local host since the TCP server port is known and the host must be local by definition.

When not creating an ET system, omit the "created" attribute. In such cases, the attributes of **eventNum**, **eventSize**, **groups**, **recvBuf**, and **sendBuf** are all ignored. Following is the simplest possible entry for attaching to an existing system since it uses all the defaults:

```
<transports>
  <server name="myEt" class="Et" />
</transports>
```

In this case the EMU needs to be run with the "-Dexpid=..." command line argument or the environmental variable EXPID needs to be defined (command line takes precedence). The reason is that if not given, the ET system name is automatically generated as /tmp/<expid>_<emu name>.

To make a direct connection to an existing ET system on a known **host** and **port**, do something like:

```
<transports>
  <server name="myET" class="Et" etName="/tmp/myET" host="myHost"
    method="direct" port="12376"/>
</transports>
```

In such cases it is meaningless to specify attributes like the number and size of events, since for an existing system they are already defined and the EMU will just ignore them.

3.1.6. Streaming TCP Socket

Similar to the cMsg emu domain, specifying parameters when **receiving** data requires 3 things:

1. the user-given **name** which will also be referred to by any channels that use it,
2. the TCP **port** number used for communication, and
3. the **class** which is always and exactly "TcpStream" (case sensitive).

Note that the xml element name is **client** since it is receiving data even though it acts, in fact, as a TCP server awaiting connections from DAQ components upstream. A unique port is required for each component in the DAQ system that uses this protocol. In jcedit the port defaults to 46100.

```
<transports>
  <client name="streamingSocket" port="46101" class="TcpStream" />
```

```
</transports>
```

When **sending** data with this transport, the port does not need to be set (since that is set in the channel) and the xml element name is **server** since it is sending data.

```
<transports>
  <server name="streamingSocket" class="TcpStream" />
</transports>
```

This code simply calls the emu domain server code underneath. See that section for more details.

3.1.7. Streaming UDP Socket

This transport is similar to the Streaming TCP transport described in the previous section. Use it by specifying 2 parameters when **receiving**:

1. the user-given **name** which will also be referred to by any channels that use it, and
2. the **class** which is always and exactly "UdpStream" (case sensitive).

Note that the xml element name is **client** since it is receiving data even though it is, in fact, a UDP receiving socket awaiting connections from DAQ components upstream. The unique port required for the receiving socket is defined in the channel.

```
<transports>
  <client name="streamingUdpSocket" class="UdpStream" />
</transports>
```

When **sending** data with this transport, the port is set in the channel and the xml element name is **server** since it is sending data.

```
<transports>
  <server name="streamingUdpSocket" class="UdpStream" />
</transports>
```

3.2. Channels

For each transport, multiple channels can be created. These are designated in the config files as **inchannel** or **outchannel** xml elements depending on which way the data flows. These elements are associated with and defined under the elements for each module. Where exactly they belong in the file will be seen later, but for now we'll only look at the individual channels.

3.2.1. FIFOs

Since FIFOs are only used between modules, they are not allowed to be specified as input to the first module and an exception will be thrown if that's the case. Although not prohibited, placing a FIFO after the last module guarantees that the data will not flow through the EMU once that FIFO is full.

In order to keep the data flow from getting ridiculously complex, if a module has a fifo as its output channel, then it may only have **ONE** output channel. Likewise, if a module has a fifo as its input channel, then it may only have **ONE** input channel.

For modules to use FIFOs to pass data between them, one does something like this:

```
<modules src="modules.jar" usr_src="user_modules.jar">
  <InModule class="someClass1">
    <inchannel id="1" name="a" transp="myFile" fileName="a" />
    <outchannel name="F1" transp="Fifo"/>
  </InModule>
  <MidModule class="someClass2">
    <inchannel name="F1" transp="Fifo"/>
    <outchannel name="F2" transp="Fifo"/>
  </MidModule>
  <OutModule class="someClass4">
    <inchannel name="F2" transp="Fifo"/>
    <outchannel id="2" name="b" transp="myFile" fileName="b" />
  </OutModule>
</modules>
```

Simply name a FIFO by using the **name** attribute of a channel and specify the **transp** attribute as being exactly "Fifo". Make one module's output channel fifo the same as the next module's input channel fifo and data will flow from one to the other.

3.2.2. Files

For a module to define a particular file as an output one does this:

```
<outchannel id="0" name="me" transp="myFile" fileName="$ (DIR) /a%d"
  split="10000" />
```

While an input file (seldom used) looks like:

```
<inchannel id="0" name="me" transp="myFile" fileName="abc" />
```

Here is a list of all the file channel attributes and what they do:

Attribute	Required	In/Out	Function	Allowed Value(s)
id		both	User given id of channel.	Any integer. Defaults to 0.
name	✓	both	User given name of channel.	Any string.
transp	✓	both	Name of the transport to use.	An already defined transport's name.
ringSize		in	Ring buffer capacity for parsed evio events from file.	Power of 2. Defaults to 4096. Min is 128.
fileName		both	Name of file to write or read.	A valid file name. May contain wildcards. For details see below. Defaults to the automatically generated name.
split		out	Number of bytes at which to create & start writing to another file.	Any non-negative integer. Defaults to 0 which means do not split the file. Negative values are ignored.
dir		out	Directory in which to write.	Valid directory. Not used by default.
dictionary		both	Name of file containing evio format xml dictionary.	A valid file name.
compression		out	Compress the output	0 = no compression (default), 1 = lz4 2 = lz4 best 3 = gzip
compressionThreads		out	Number of threads used to do data compression	> 0. Defaults to 1.
evioRamBuffer		out	Bytes specified for each of the evio writer's internal buffers	Minimum value of and defaults to 64Mbytes. 3 buffers of this size are created.

The most complicated part of the file channel is the default values of and the automatic generating of the file name. Let's start with splitting the file. In order to prevent output files from getting too large, they can be split into smaller ones. This happens when the user specifies a positive value for the ***split*** attribute which specifies the maximum file size in bytes before it is closed and the next one created. A negative or zero value means no splitting takes place. When files are split, each created file has a different integer number appended to or placed somewhere in its name to differentiate it from the previous one. This number starts at 0 and increases by 1 for each subsequent file and is called the ***split count***.

The rules for automatic naming of files are built into evio and are as follows. The base file name may contain up to 3, C-style integer format specifiers using "d" and "x" (such as %03d, or %x). If more than 3 are found, an exception will be thrown. If no "0" precedes any integer between the "%" and the "d" or "x" of the format specifier, it will be added automatically in order to avoid spaces in the generated name. The first occurrence will be substituted with the given run number. If the file is being split, the second will be

substituted with the split count. If there are multiple streams, the third will be substituted with the stream id (this id is given to the emu by run control when there are multiple, parallel data streams).

If no specifier for the split count exists, it is tacked onto the end of the file name. If no specifier for the stream id exists, it is tacked onto the end of the file name, after the split count. No run numbers are ever tacked on without a specifier.

For splitting: if there is only 1 stream, no stream ids are used and any third specifier is removed.

For non-splitting: if there is only 1 stream, no stream ids are used and any second and third specifiers are removed. For multiple streams, the second specifier is removed and the 3rd substituted with the stream id.

For all cases: if there are more than 3 specifiers, NO SUBSTITUTIONS ARE DONE.

The base file name may contain characters of the form “\$(ENV_VAR)” which will be substituted with the value of the associated environmental variable or a blank string if none is found. It may also contain occurrences of the string "%s" which will be substituted with the value of the run type or nothing if the run type is null.

If no fileName is given, for input files it defaults to reading from codaDataFile.evio. For output files, it defaults to writing to <session>_<run#>.dat<file_count> where session refers to its session name received from run control. If **dir** is defined then it writes to a file of the generated name in that directory and it reads codaDataFile.evio from that directory as well.

If, on the other hand, fileName is given, then the first thing that happens is that all substitutions mentioned above are made. For input files, it reads from that file (in the directory dir if given). For output files, it writes to that file (in the directory dir if given).

If this is all too stinkin' complicated, play with the evio org.jlab.coda.jevio.Utilities.generateBaseFileName () method followed by calling the generateFileName() method. The first provides input for the second and the second returns the final file name. See the evio javadocs.

3.2.3. cMsg – cMsg domain

For a module to define a particular cMsg server as an output one does this:

```
<outchannel id="0" name="me" transp="mycMsg subject="a" type="b" />
```

While an input cMsg subscription looks like:

```
<inchannel id="0" name="me" transp="mycMsg" subject="sub" type="*" />
```

Here is a list of all the cMsg channel attributes and what they do:

Attribute	Required	In/Out	Function	Allowed Value(s)
id		both	User given id of channel.	Any integer. Defaults to 0.
name	✓	both	User given name of channel.	Any string.
transp	✓	both	Name of the transport to use.	An already defined transport's name.
ringSize		in	Ring buffer capacity for parsed evio events from cMsg.	Power of 2. Defaults to 4096. Min is 128.
single		out	If true, each evio event sent out in single cMsg msg, else events marshalled into 1 msg.	Case indep. "true", "on", or "yes". Anything else is false. Default = false
subject		both	cMsg subscription subject for inchannel. cMsg message subject for outchannel.	Valid cMsg subject. Subscription subject may contain wildcard chars.
type		both	cMsg subscription type for inchannel. cMsg message type for outchannel.	Valid cMsg type. Subscription type may contain wildcard chars.
wthreads		out	Number of cMsg message buffer filling threads	Positive int which defaults to 1. Max is 10.

A note on the input channel's subject and type. In the cMsg domain, the wildcard characters "*", "?", and "#" are allowed in subscriptions' subject and type, where "*" matches any number of characters, "?" matches a single character, and "#" matches one or no positive integer. Also, wildcard constructs like {i>5 | i=4} can be used to match a range of positive integers which meet the conditions in the parentheses. The logic symbols >, <, =, |, & are allowed along with the letter i, any positive integers, and spaces. As an example, a subscription to the subject abc{i>22 & i<26} will match a message with the subjects abc23, abc24, and abc25.

3.2.4. cMsg – emu domain

For a module to define a particular emu socket output channel one does something like:

```
<outchannel id="0" name="me" transp="myEmu timeout="5" port="46123"
maxBuf="256000" />
```

While an emu socket input channel looks like:

```
<inchannel id="0" name="me" transp="myEmu" />
```

Here is a list of all the emu domain channel attributes and what they do:

Attribute	Required	In/Out	Function	Allowed Value(s)
id		both	User given id of channel.	Any integer. Defaults to 0.
name	✓	both	User given name of channel.	Any string.
transp	✓	both	Name of the transport to use.	An already defined transport's name.
ringSize		in	Ring buffer capacity for parsed evio events from cMsg.	Power of 2. Defaults to 4096. Min is 128.
single		out	If true, each evio event sent out immediately over socket, else events marshalled before sending.	Case indep. "true", "on", or "yes". Anything else is false. Default = false
recvBuf		in	TCP socket receive buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
sendBuf		out	TCP socket send buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
noDelay		out	TCP NODELAY value.	Case indep. "true", "on", or "yes". Anything else is false. Default = false.
subnet		out	If is a local IP address, it is used for outgoing traffic. If subnet address, local IP address on that subnet is chosen, if any.	Dotted-decimal format IP address.
maxBuf		out	Internal buffer size for sending.	Max size in bytes of a single send and defaults to 2.1MB.
port		out	UDP port to multicast to when finding emu server or TCP port for direct connection	Integer > 1023 and < 65536. Defaults to 46100.
timeout		out	Time in seconds to wait for connecting to emu server	Any non-negative integer. Defaults to -1 (ignore).
sockets		both	Number of TCP sockets to move data in each channel.	> 0. Defaults to 1.

In download, the emu domain transport object will have created a server for the downstream component, say an EB. In prestart the upstream component, say a ROC, module will create its output channel by making a TCP connection to this server. The UDL used to make the connection is:

```
emu://port/expid/compName?codaId=id&timeout=timeout&bufSize=maxBuf&tcpSend=sendBuf
&subnet=subnet&noDelay
```

where “port” is the TCP port to use when creating the socket, “compName” is the name of the destination CODA component, “timeout” is the time to wait in seconds for connecting to emu server and defaults to 3 seconds, “bufSize” is the max size in bytes of a single send and defaults to 2.1MB, “tcpSend” is the TCP send buffer size in bytes, “subnet” is the preferred subnet (broadcast) IP address used to connect to server, and “noDelay” is the TCP no-delay parameter turned on.

3.2.5. ET

For a module, for example an EMU-based ROC, sending output to a particular ET system:

```
<outchannel id="1" name="EB" transp="myET" capacity="24" group="1" chunk="6"
           recvBuf="350000" sendBuf="350000" noDelay="on" />
```

While input from an ET system, for an EB for example, looks like:

```
<inchannel id="1" name="Roc1" transp="myEt" stationName="stat1"
           position="1" idFilter="on" chunk="1"/>
```

Here is a list of all the ET channel attributes and what they do:

Attribute	Required	In/Out	Function	Allowed Value(s)
id		both	User given id of channel.	Any integer. Defaults to 0.
name	✓	both	User given name of channel.	Any string.
transp	✓	both	Name of the transport to use.	An already defined transport's name.
ringSize		in	Ring buffer capacity for parsed evio events from ET.	Power of 2. Defaults to 4096. Min is 128.
single		out	If true, each evio event sent out in single ET buffer, else evio events marshalled into 1 ET buffer.	Case indep. "true", "on", or "yes". Anything else is false. Default = false.
revBuf		both	ET consumer's TCP receive buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
sendBuf		both	ET consumer's TCP send buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
noDelay		both	ET consumer's TCP NODELAY value.	Case indep. "true", "on", or "yes". Anything else is false. Default = false.
group		out	Group from which to obtain new (unused) events.	Integer > 0. Defaults to 1.
chunk		both	How many events to get at once (in an array).	Integer > 0. Defaults to 100.
stationName		in	Name of station to attach to and to create if non-existing.	Any string with < 48 characters. If inchannel, defaults to station<id> . If outchannel, GRAND_CENTRAL is used.
position		in	Set position of existing or created station.	Integer > 0. Defaults to 1. GRAND_CENTRAL has reserved position of 0.
idFilter		in	Station only accepts events from component with this coda id.	Case indep. "on", everything else is off. Default is off.
controlFilter		in	Allow only control events into station. Overrides idFilter.	Case indep. "on", everything else is off. Default is off.
prescale		out	Used only in ER. Any ET output channel receives a prescaled number of the physics events (i.e. if prescale = 3, only every 3 rd event is sent over the channel).	> 0. Defaults to 1.
ignoreDataErrors		in	Any error reading/parsing data can be ignored if true. Currently this is only implemented for ER's ET input channel if the main data is coming over an emu socket input channel.	Case indep. "true", "on", or "yes". Anything else is false. Default = false.

3.2.6. Streaming TCP Socket

For a module to define a particular TCP streaming socket output channel one does something like:

```
<outchannel id="8" name="Dc" transp="StreamingTcp" timeout="5" port="46100"
maxBuf="90000" />
```

While an input channel looks like:

```
<inchannel id="8" name="Dc" transp="StreamingTcp" streams="1" />
```

Here is a list of all the TCP streaming channel attributes and what they do:

Attribute	Required	In/Out	Function	Allowed Value(s)
id		both	User given id of channel.	Any integer. Defaults to 0.
name	✓	both	User given name of channel.	Any string.
transp	✓	both	Name of the transport to use.	An already defined transport's name.
ringSize		in	Ring buffer capacity for parsed evio events.	Power of 2. Defaults to 4096. Min is 128.
single		out	If true, each evio event sent out immediately over socket, else events marshalled before sending.	Case indep. "true", "on", or "yes". Anything else is false. Default = false
recvBuf		in	TCP socket receive buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
sendBuf		out	TCP socket send buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
noDelay		out	TCP NODELAY value.	Case indep. "true", "on", or "yes". Anything else is false. Default = false.
subnet		out	If is a local IP address, it is used for outgoing traffic. If subnet address, local IP address on that subnet is chosen, if any.	Dotted-decimal format IP address.
maxBuf		out	Internal buffer size for sending.	Max size in bytes of a single send and defaults to 4MB.
port		out	UDP port to multicast to when finding server or TCP port for direct connection	Integer > 1023 and < 65536. Defaults to 46100.
timeout		out	Time in seconds to wait for connecting to server	Any non-negative integer. Defaults to -1 (ignore).
streams		in	Number of streams from one VTP-based ROC to DC/SEB	Integer of value 1,2,3, or 4. Defaults to 1.

Since the emu domain server is used underneath, see the section on the emu domain channel for more details. In this case, there is no option for a “fat” channel which uses multiple sockets for one connection.

A note on the complicated “streams” attribute. This attribute is not parsed in the channel’s constructor as the others are. It’s parsed in the prestart method of the EMU. The reason for this is that a single VTP-based ROC can have 1 - 4 streams and thus the receiving CODA component must have the same number of corresponding input channels. Yet when configuring, it appears as 1 channel in jcredit. Thus, behind the scenes in the EMU we may have to generate channels – 1 for each stream. This attribute warns the EMU that these channels need to be created in prestart. If there are multiple streams, the port becomes a “starting” port number. Each generated channel’s port is incremented by 1 from the previous.

In addition, if in the EMU’s prestart method it finds that all inputs channels are from a single VTP, when time slices are built, it merges them into a single time slice from that VTP (as opposed to treating them like separate VTPs).

3.2.7. Streaming UDP Socket

For a module to define a particular UDP streaming socket output channel one does something like:

```
<outchannel id="8" name="Dc" transp="streamingUdp" port="17777"
bufSize="89000" host="127.0.0.1" useErsapReHeader="true" />
```

While an input channel looks like:

```
<inchannel id="8" name="Roc" transp="streamingUdp" port="17777"
bufSize="90000" streams="2" useErsapReHeader="true" />
```

Here is a list of all the UDP streaming channel attributes and what they do:

Attribute	Required	In/Out	Function	Allowed Value(s)
id		both	User given id of channel.	Any integer. Defaults to 0.
name	✓	both	User given name of channel.	Any string.
transp	✓	both	Name of the transport to use.	An already defined transport's name.
ringSize		in	Ring buffer capacity for parsed evio events.	Power of 2. Defaults to 4096. Min is 128.
single		out	If true, each evio event sent out immediately over socket, else events marshalled before sending.	Case indep. "true", "on", or "yes". Anything else is false. Default = false
recvBufSize		in	UDP socket receive buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
sendBufSize		out	UDP socket send buffer byte size.	Integer >= 0. Value of 0 gives operating system default. Defaults to 0.
bufSize		both	Byte size for each buffer in supply used for reconstructing incoming or building outgoing data buffers.	Initial size in bytes of a single buffer. Defaults to 100kB (will expand as needed).
port		both	UDP socket port to either send or receive on	Integer > 1023 and < 65536. Defaults to 46100.
host	✓	out	Host to send packets to	Any string.
useErsapHeader		both	If true, use the packet ERSAP/EJFAT header, else the CODA header	Case indep. "true", "on", or "yes". Anything else is false. Default = false
useLoadBalancer		out	If true, insert packet header for using EJFAT load balancer	Case indep. "true", "on", or "yes". Anything else is false. Default = false
streams		in	Number of streams from one VTP-based ROC to DC/SEB	Integer of value 1,2,3, or 4. Defaults to 1.

A note on the complicated “streams” attribute. This attribute is not parsed in the channel’s constructor as the others are. It’s parsed in the prestart method of the EMU. The reason for this is that a single VTP-based ROC can have 1 - 4 streams and thus the receiving CODA component must have the same number of corresponding input channels. Yet when configuring, it appears as 1 channel in jcredit. Thus, behind the scenes in the EMU we may have to generate channels – 1 for each stream. This attribute warns the EMU that these channels need to be created in prestart. If there are multiple streams, the port becomes a “starting” port number. Each generated channel’s port is incremented by 1 from the previous.

In addition, if in the EMU’s prestart method it finds that all inputs channels are from a single VTP, when time slices are built, it merges them into a single time slice from that VTP (as opposed to treating them like separate VTPs).

3.2.8. Event Recorder – special rules

There are a number of special rules that apply to the Event Recorder's handling of channels. There is no restriction on a single input channel. However, there should never be more than 2 input channels in which case one must be an emu socket and the other an ET channel. The emu socket is assumed to carry the main flow of physics events. Any ET input channel is assumed to carry user events and is given a lower priority. This means reading from it should never block.

The only output channel types allowed are ET and file. A maximum of 1 ET output channel is permitted. All control and “first” events are sent over all channels. Any “first” event coming before the prestart event is placed after it instead. User events, however, are placed only into the first file channel. If no file channels exist, they're placed into the ET channel. A prescaled number of output physics events are sent over the ET channel. Whereas physics events are sent round-robin to all file channels.

Chapter 4

4 Modules

Modules are the heart of an EMU and perform all the data processing tasks. While modules can theoretically be written by anyone, in practice it is not a simple thing to do and is best left to the Jefferson Lab DAQ group. This chapter covers the modules written and supported by the DAQ group. Although it is possible to string multiple modules together in a single EMU, this capability is currently unneeded and unused. Each EMU used in the DAQ system contains a single module.

Modules are implemented as objects created from dynamically loaded Java classes. One benefit of this design is that module behavior can be changed in fully operational CODA DAQ systems without stopping to recompile and restart the software, facilitating quick software development. During the download transition, all existing modules are removed and new ones are created. Thus all the user needs to do is to modify any module code, compile it, then use run control to issue a download command. The newly modified module will be used automatically.

The JLAB DAQ group provides the modules necessary for creating a functional data acquisition system. These include modules to implement: 1) an event builder, 2) an event recorder, 3) a simulated ROC, 4) a simulated trigger supervisor, and 5) a farm controller.

4.1. Config File

An EMU's configuration file has 2 sections. The previous chapter dealt with the first section on transports. This chapter deals with the second section on modules. The xml element used is not surprisingly named **modules**. It is used to configure all the modules used in a single EMU. The following is an example from a config file for an event builder.

```
<modules>
  <EbModule class="EventBuilding" id="2" timeStats="off" runData="false"
    tsCheck="true" tsSlop="2" sparsify="false" >

    <inchannel id="0" name="Roc1" transp="inET" idFilter="on" />
    <inchannel id="2" name="Roc2" transp="inET" idFilter="on" />
    <outchannel id="4" name="Er1" transp="outET" group="1" chunk="5" />
  </EbModule>
</modules>
```

The classes used to implement the supported modules are all included the emu jar file, emu-3.3.jar. This jar file must be in the user's CLASSPATH environmental variable in order for Java to find them. If a user-supplied class is being used instead, its full name

must be specified in the class attribute in the xml element defining the module (***EbModule*** in the example above). The jar file which contains it must also be in the user's CLASSPATH.

Under the <modules> element are elements for each of the modules to be loaded. Modules' element names (***EbModule*** in the above example) are used only for readability and may be set by the user to any string. Attributes for each module are what determines its behavior and depend entirely upon the module itself. Currently, all DAQ-provided modules only have ***inchannel*** and ***outchannel*** subelements to determine its input and output data channels.

4.2. Fast Event Builder

The event building module implements all the functionality of an event builder. This module can function as a data concentrator (DC) which is the first level builder of a 2-stage event building. It can function as a stage event builder (SEB) which is the second level of a 2-stage event building. It also can function as a primary event builder (PEB) which is a single, stand-alone event builder that does a complete build. A rough outline of this module's internal structure is given in Figure 4.1 below.

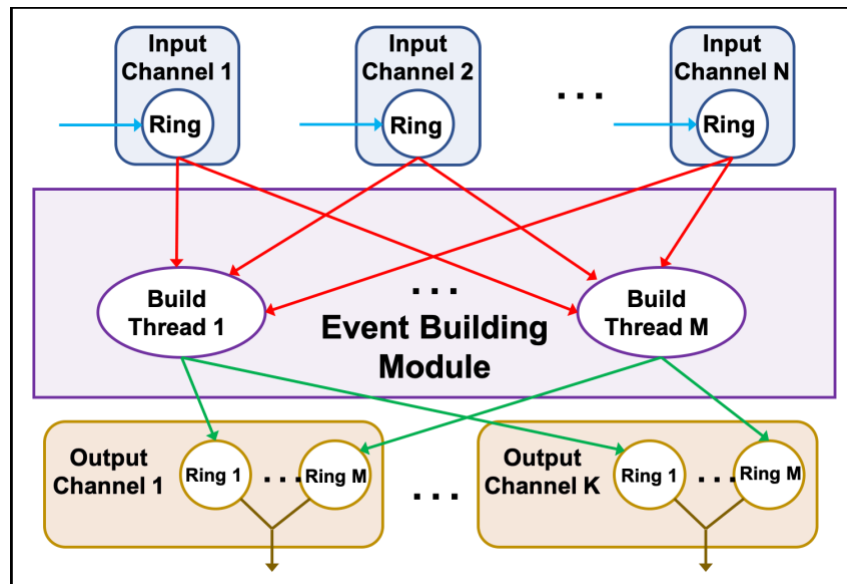


Figure 4.1 Event Builder Internals

Everything inside the center purple box in the figure above represents the event building module. Rings are fast circular buffers which take the place of queues used in previous versions of this software package.

If this module is used to build ROC raw events, each ROC will have its own input channel. If it's used as a second-level event builder, each first-level EB will have its own input channel. In any case, a single event from each input channel will be needed to build a complete event.

This is accomplished by the building threads - the number of which is determined by the config file. If not explicitly set it defaults to 2. Each thread will grab an event from each channel's ring, build them into a single event and place that built event on an output channel's ring. If more than one output channel exists, the physics events are placed in output channels in round-robin fashion.

User events are not built but simply passed on to the first ring of the first output channel. Any control events must appear on each input ring in the same position. If not, an exception is thrown. If so, the control event is passed along to all output channels. If no output channels are defined in the config file and no additional modules follow it, this module discards all built events.

The user should be aware that this module can act as an event recorder in addition to its ability to build. Simply set its output channel to be a file.

Some of the details of the event building can be controlled through the configuration file. Following is a list of the module's attributes that can be set:

Attribute	Required	Function	Allowed Value(s)
id		CODA id of event builder	Any non-negative integer. Defaults to 0.
class	✓	Name of Java class defining this module.	Must be exactly "EventBuilding".
threads		Number of event building threads.	Any positive integer. If not given, EB will have 2.
endian		Endianness of output data.	Set to case indep. "little" for little endian, else big. Default is big.
repStats		Does this module's stats accurately represent the whole EMU?	Case indep. "false", "off", or "no". Anything else is true. Default = true.
timeStats		Make histogram of time to build single events (in nsec). Printed in console during END transition.	Yes, true, on (case insensitive) to make histogram. Default is off since it kills performance.
ringCount		Number of internal, reusable ByteBuffers in which to place built events in each build thread.	Will be made a power of 2. Default = 256/threads (64 for 3 thds), min 16.
tsCheck		Check consistency of timestamps. Throw exception if inconsistent.	No, false, off (case insensitive) to not check. Default is on.
tsSlop		Maximum allowed differences (slop) in timestamps in ticks.	Any positive integer, ignores other values. Defaults to 2.
sparsify		If on, do not include roc-specific segments in trigger bank.	Yes, true, on (case insensitive) to sparsify. Default is off.
runData		Include run number and run type in built trigger bank.	Yes, in, true, on (case insensitive) to include data. Default is off.
releaseThd		Create 1 post-build thread per in channel used to release its ring slots in sequence. Obsolete since now using sequential release ByteBufferSupply.	Yes, in, true, on (case insensitive) to include data. Default is off. Keep this off.

4.3. Stream Aggregator

The stream aggregator module performs the same function as the event building module but does it for data in the streaming format which is different than the triggered data format. This module can function like a data concentrator (DC) as a first level builder of a 2-stage time slice building. It can also function like a secondary event builder (SEB) as a second level of a 2-stage time slice building. Finally, it can function like a primary event builder (PEB) as a single, stand-alone time slice builder that does a complete build. A rough outline of this module's internal structure is given in Figure 4.2 below:

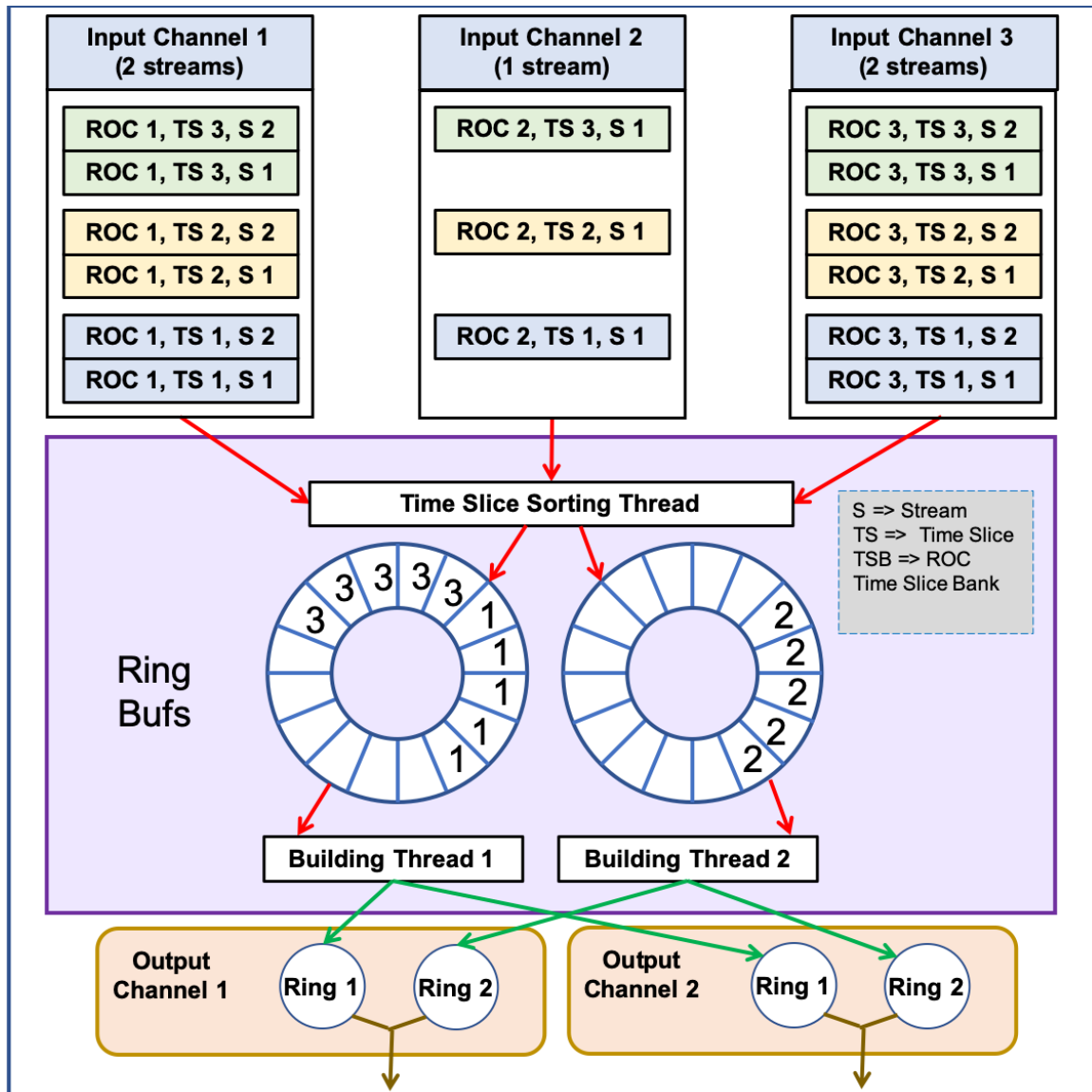


Figure 4.2 Stream Aggregator Internals

Everything inside the center purple box in the figure above represents the stream aggregating module. Rings are fast circular buffers which take the place of queues used

in previous versions of this software package. And in this case, the input channels contents are contained in those circular buffers as well.

Streaming data comes in time slices. Each input channel has many time slices coming in and the Time Slice Sorting Thread distributes all slices from a single time to one build thread. There may be as many build threads as desired. Each build thread will construct a full time slice which it then passes to an output channel. If more than one output channel exists, the slices are placed in output channels in round-robin fashion.

If no input channels have a particular timestamp, in other words all data is missing for a particular frame, then the aggregator will create and insert an “event” which represents this empty frame into the data stream.

User events are not built but simply passed on to the first ring of the first output channel by the sorting thread. Likewise, that thread sends control events to the output channels. The END event is handled separately. All control events are passed along to all output channels. If no output channels are defined in the config file and no additional modules follow it, this module discards all built buffers.

Some of the details of the time slice building can be controlled through the configuration file. Following is a list of the module's attributes that can be set:

Attribute	Required	Function	Allowed Value(s)
id		CODA id of event builder	Any non-negative integer. Defaults to 0.
class	✓	Name of Java class defining this module.	Must be exactly "Aggregator".
threads		Number of time slice building threads.	Any positive integer. If not given, will have 2.
endian		Endianness of output data.	Set to case indep. “little” for little endian, else big. Default is big.
repStats		Does this module’s stats accurately represent the whole EMU?	Case indep. "false", "off", or "no". Anything else is true. Default = true.
timeStats		Make histogram of time to build single events (in nsec). Printed in console during END transition.	Yes, true, on (case insensitive) to make histogram. Default is off since it kills performance.
ringCount		Number of internal, reusable ByteBuffers in which to place built time slices in each build thread.	Will be made a power of 2. Default = 256/threads (64 for 3 thds), min 16.
tsSlop		Maximum allowed differences (slop) in timestamps in ticks.	Any positive integer, ignores other values. Defaults to 2.

4.4. Event Recording

One benefit of abstracting out the data communication from the modules is that the event recorder becomes trivial to implement. All the real work is done in the I/O transports and channels. The event recording module uses a single thread to funnel all input events into all of the output channels.

There are a number of special rules that apply to the Event Recorder's handling of channels. There is no restriction on a single input channel. However, there should never be more than 2 input channels in which case one must be an emu socket and the other an ET channel. The emu socket is assumed to carry the main flow of physics events. Any ET input channel is assumed to carry user events and is given a lower priority. This means reading from it should never block.

The only output channel types allowed are ET and file. A maximum of 1 ET output channel is permitted. All control and "first" events are sent over all channels. Any "first" event coming before the prestart event is placed after it instead. User events, however, are placed only into the first file channel. If no file channels exist, they're placed into the ET channel. A prescaled number of output physics events are sent over the ET channel.

Whereas physics events are sent round-robin to all file channels.

Following is a list of the module's attributes that can be set:

Attribute	Required	Function	Allowed Value(s)
id		CODA id of event recorder	Any non-negative integer. Defaults to 0.
class	✓	Name of Java class defining this module.	Must be "EventRecording" which is the Java class name.
repStats		Does this module's stats accurately represent the whole EMU?	Case indep. "false", "off", or "no". Anything else is true. Default = true.

4.5. ROC Simulation

This was written merely for testing purposes. In cases when an actual data-producing ROC is unavailable, this module will provide an EMU-based ROC which generates ROC raw records. Having such a ROC allows testing of CODA components downstream such as event builders and event recorders.

The data in each record/event are all 1's except the very first word which is the event number. There is an option to use real data from Hall D. There are 9 files, each containing 16MB of data which is enough to fill the entire ring buffer with data for a more realistic simulation or a single ROC. This option is set by hand editing the **useRealData** variable in ROCSimulation.java's constructor. It's set to true by default. The data-loading algorithm will try to match the data file name's ending number with the ROC name's ending number. Thus, ROC1 – ROC9 can each have unique data. You may have to hack it to get it work for you.

As an aside, since it's too much work to have jccedit and config files modified each time a new variable is added to modify behavior, the normal mode of operation is to edit this module's code and make another the jar file for testing purposes.

In order for any simulation to work properly, all ROCs must have sent the same number of events when the run control commands to end or reset are given. This is done by having ROCs sync with each other through a simulated Trigger Supervisor every 20k events through sending cMsg messages - a poor man's trigger. For more detail, go [here](#). ROC emus are automatically set to sync by means of a TsSimulation (trigger supervisor)

emu. If you wish to run without the simulated TS, which works fine if there's only 1 ROC, add

sync="off"

to the ROC module's config file or else your ROCs will sit around all day waiting to communicate with the TS.

There are some parameters the user can control in the following table:

Attribute	Required	Function	Allowed Value(s)
id		CODA id of ROC	Any non-negative integer. Defaults to 0.
class	✓	Name of Java class defining this module.	Must be "RocSimulation" which is the Java class name.
threads		Number of data generating threads.	Positive integer. Defaults to 1. Values < 1 get set to 1.
triggerType		Trigger type from trigger supervisor.	Integer between 0 and 15 inclusive. Defaults to 15. Negative values set it to 0, over 15 gets set to 15.
detectorId		Id of detector producing data in data block bank.	Integer ≥ 0 . Defaults to 111. Negative values set it to 0.
blockSize		Number of events in one (entangled) data block.	Integer between 1 and 255 inclusive. Defaults to 40. Values < 1 set it to 1, over 255 gets set to 255.
eventSize		Number of bytes in a single event	Positive integer. Defaults to 75, min = 1
syncCount		Number of writes of a single, entangled block of evio events, before syncing with other simulated ROCs.	Positive integer. Defaults to 20k, min is 10.
sync		Do we sync this with other simulated ROCs?	Case indep. "false", "off", or "no". Anything else is true. Default = true.

4.6. Trigger Supervisor Simulation

This was written merely for testing purposes. In cases when actual data-producing ROCs are unavailable, this module will provide a trigger for EMU-based ROCs. Having such a system allows testing of CODA components downstream such as event builders and event recorders.

In order for any simulation to work properly, all ROCs must have sent the same number of events when the run control commands to end or reset are given. This is done by having ROCs sync with each other through this simulated Trigger Supervisor every 20k events by sending cMsg messages - a poor man's trigger. For more detail, go [here](#).

The TS config file must contain a list of ROCs to synchronize as attributes as in the example below:

```
<modules>
  <TsModule class="TsSimulation" r1="Roc1" r2="Roc7" />
```

</modules>

Starting with r1 and going up sequentially in number, each ROC name is identified. These will be the ROCs that will be synchronized to each other. The following are parameters that can be set:

Attribute	Required	Function	Allowed Value(s)
id		CODA id of trigger supervisor	Any non-negative integer. Defaults to 0.
class	✓	Name of Java class defining this module.	Must be "TsSimulation" which is the Java class name.
rN	✓	Identify all ROCs to sync.	"r" followed by a sequential number (starting with 1) equals ROC's name. Use as many times as there are ROCs (e.g. r1="Roc1" r2="Roc7" r3="myRoc").

4.7. Farm Controller

This is a simple emu designed to work with an ET system for input and to pass events right through much as an ER. The farm controller is set up to consume only control events from its input channels (hopefully only one channel) and pass it through to all output channels (also hopefully only one). The ET input channel needs to have the attribute:

```
controlFilter="on"
```

in order to accept only control events. The physics events are all handled by the farm nodes.

Attribute	Required	Function	Allowed Value(s)
id		CODA id of farm controller.	Any non-negative integer. Defaults to 0.
class	✓	Name of Java class defining this module.	Must be "FarmController" which is the Java class name.

Chapter 5

5 Running an EMU with Run Control



Figure 5.1 A running emu

Emus run pretty fast. When they reach their top speed, we say they're smokin'.



Figure 5.2 A smokin' emu

The running of an EMU is very easy. Configuring it with *jccedit* may be challenging, but actually running it is simple. Although several can be run in a JVM simultaneously, in CODA only 1 emu is ever run in a single JVM.

5.1. Config File Final Form

Each configuration file may specify a single EMU to be run in a single JVM. In outline form it looks like:

```

<?xml version="1.0"?>
<component name="EB" type="SEB" >
  <transports>
    ...
  </transports>
  <modules>
    ...
  </modules>
</component>

```

The previous 2 chapters explain in some detail the nature of the xml entries under transports and modules. The **component** element has 2 attributes, **name**, which must be present and unique in run control, and also **type** which is the CODA type and is optional (see [Appendix A](#) for values allowed).

5.2. Creating EMUs

EMUs are created using the **EmuFactory** class which does the work of sorting through command line arguments, reading and parsing config files, and starting up all the EMUs that are indicated in these arguments. To run it execute,

```
java org.jlab.coda.emu.EmuFactory
```

where the following options are allowed and the “-D” options need to precede the class name:

Argument	Required	Function
-h or -help		Print help.
-Dname		Name of CODA component to create. Can be list of names separated by “,”, “:”, or “;” for multiple components.
-Dtype		Type of CODA component to create. See Appendix A . Can be list of types corresponding to list of names separated by “,”, “:”, or “;” for multiple components.
-DcmsgUDL	✓	cMsg UDL used to connect to the cMsg server the AFECS platform is running.
-Dexpid		Set the experimental id, overriding environmental variable EXPID in EMU. If not set on cmd line or in env.var., default to “unknown”. Used to generate default ET system name (/tmp/<expid>_<emuName>). Used to construct default UDL (rc://multicast/<expid>) to connect to platform cMsg server if cmsgUDL not defined .
-Dsession		Set the experimental session. Used to generate output file names. Run control, if being used, sets the session, so this is useful only when using the EMU with the debug gui.
-DDebugUI		Start up a debug gui to run EMU without run control.
-Duser.name		Arg passed to EMU objects which set user's for use in error messages.

5.3. Platform connection

In order to place an EMU under run control, it must be able to connect to the desired AFECs platform. This is done using the **cmsgUDL** option to specify the run control domain cMsg server that's running inside the platform. That sounds complicated, but in practice the user only needs to type:

```
java -DcmsgUDL="rc://multicast/<expid>" org.jlab.coda.emu.EmuFactory
```

where **<expid>** is replaced by the user's run control experiment id. This enables the necessary communication. If the EMU is run but the platform is down, the EMU will continue to try to connect until the platform comes up and it succeeds.

5.4. Running a single EMU

The only way to configure an EMU is by using the **jcredit** program to create config files which are then passed to components by run control during the **configure** transition. However, the user must provide the EMU's name on the command line, otherwise there is no way for run control to know which configuration to send it. In addition, it is also necessary to specify the component type on the command line. Although it is possible to specify the type in the config file, jcredit currently does not do it. Do something like the following to use run a single emu:

```
java -Dname=Eb -Dtype=SEB -DcmsgUDL="rc://multicast/myExpid"
    org.jlab.coda.emu.EmuFactory
```

5.5. Running Multiple EMUs

It is possible, for whatever reason, to run multiple EMUs in a single JVM. To do this the names and types are specified as single strings with the individual components separated by either colons, semicolons, or commas but **not** white space. Here's an example:

```
java -Dname=Roc,Eb,Er -Dtype=ROC,PEB,ER -DcmsgUDL="rc://multicast/myExpid"
    org.jlab.coda.emu.EmuFactory
```

This will run 3 EMUs in 1 JVM – a fake ROC, an EB and an ER.

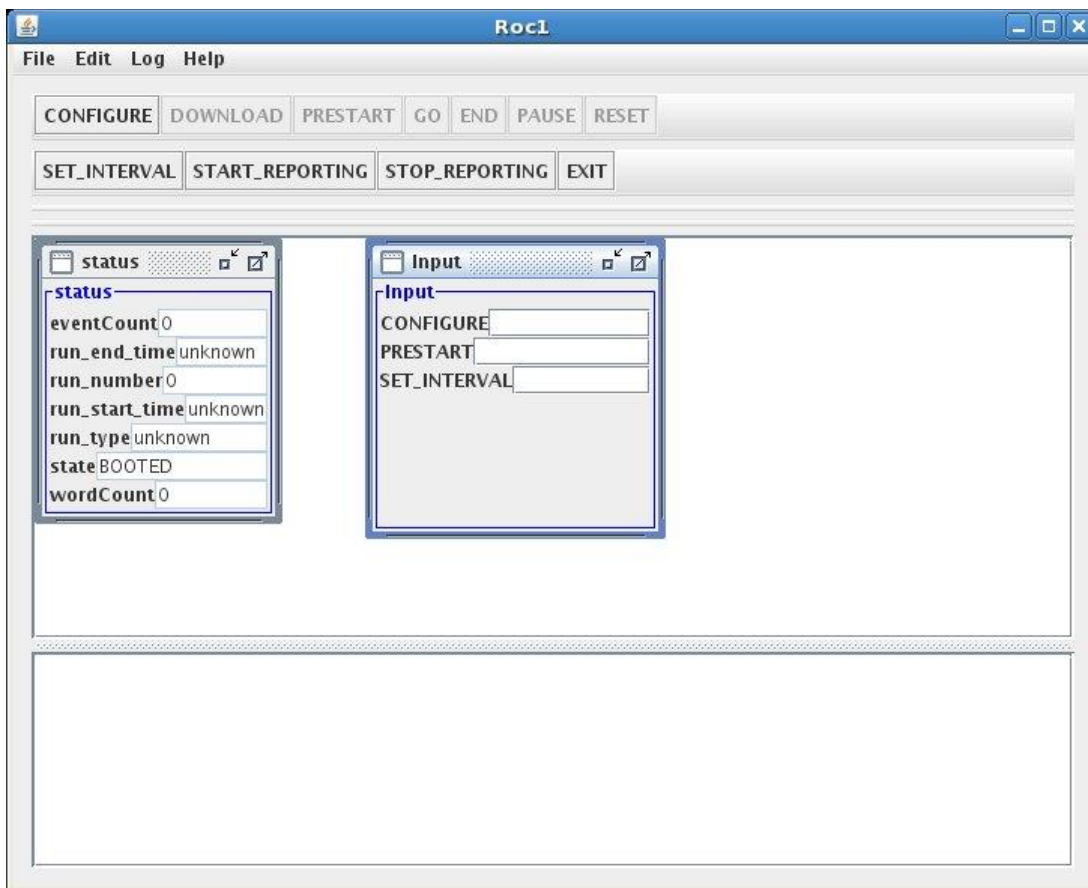
Chapter 6

6 Running an EMU with the Debug GUI

EMUs can be run standalone - without run control - which may not appear to make much sense on the surface of things since they are designed to respond to run control commands. However, there is a gui used for debugging which is part of the EMU and can be run by specifying **-DDebugUI** on the command line:

```
java -Dname=Roc1 -DDebugUI org/jlab/coda/emu/EmuFactory
```

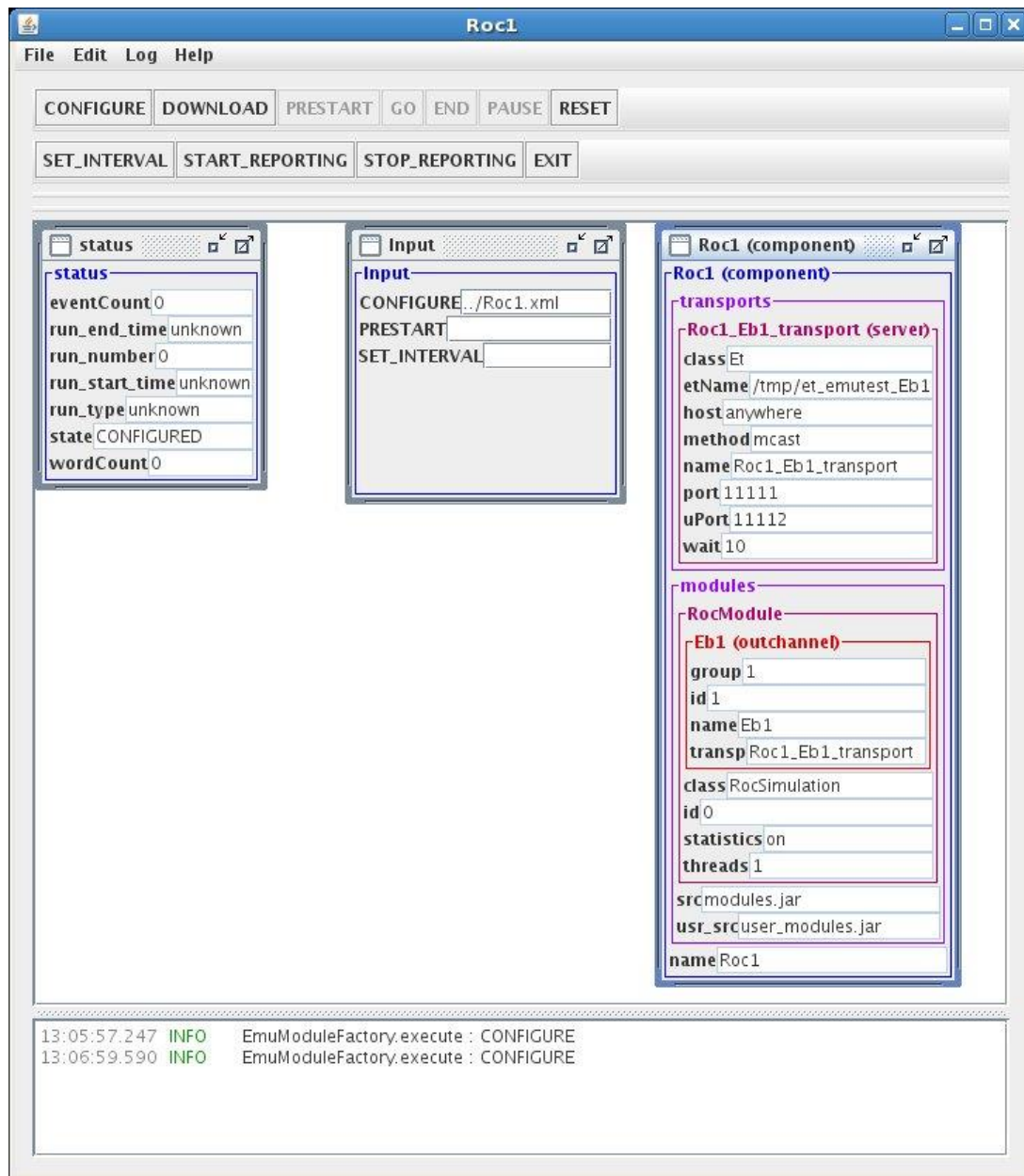
This gui allows the user to send locally generated run control commands to the EMU and see its output. It looks like:



The top panel of the application has a number of run control transition-inducing buttons along with a few other run control commands. In the bottom panel is a listing of all the EMU-produced debugging, info, warning, and error messages.

In the middle, the status window keeps track of a few stats and the input window allows user input into the EMU. Type the desired config file name in the CONFIGURE input widget and hit the CONFIGURE button to get it to load that file. The input to PRESTART is the run number. And the input to the SET_INTERVAL sets the time interval between status messages in seconds.

Once the configure button is pressed and the configuration is loaded, a new window appears:



The new window simply shows what is in the loaded configuration. In this case there is one ET transport with its name, host, connection method, ports, etc. specified. The module to be loaded is also specified - the ROC simulation module. Under modules, its output channel, the ET system, can also be seen. Notice the EMU's messages in the bottom panel.

If running multiple EMUs in 1 JVM, there will be one gui per component.

Chapter 7

7 Developer's Details

There are many details concerning the internals of the EMU which are of no interest to the general user. This chapter contains some such detail more as a reminder to the EMU developer. Although comments are spread through the code itself, it's always nice to have more coherent notes on the software that serve to jog the memories that lose much of the finer points over time.

7.1. *cMsg Run Control Connection*

There's a cMsg connection in the rc domain maintained for receiving and responding to run control commands as well as for sending dalogmsg messages. Generally the rc multicast server being connected to resides in the AFECS platform. The general form of the UDL used to connect to it is:

```
rc://<host>:<port>/<expid>?connectT0=<timeout>&ip=<address>
```

where:

- **host** is required and may also be "multicast", "localhost", or in dotted decimal form
- **port** is optional with a default of 45200
- **expid** is the run control experiment id being used for the experiment currently underway
- **timeout** is the time to wait in seconds before connect returns a timeout while waiting for the rc server (in AFECS platform) to send a special (TCP) concluding connect message. Defaults to 30 seconds.
- **address** is the IP address in dot-decimal format which the rc server or agent must use when connecting to this rc client. This is sent to the rc server along with its corresponding subnet address. If this is not set, then the rc client sends the rc server all of its IP addresses and subnet addresses.

This UDL may be specified by the user when starting up the emu by giving the following flag to the JVM:

```
-DcmsgUDL=<udl>
```

This is done for the user in the provided emu startup scripts with the value:

```
-DcmsgUDL=rc://multicast/$EXPID
```

which is also the UDL used if not given when starting up an EMU directly.

7.2. *Data Flow*

In some sense the EMU is like a mini DAQ system in one program. To refresh your memory a traditional DAQ originates data in a ROC, it flows through the EB and eventually finishes with the ER storing it somewhere. To properly shut down such a system, run control first informs the ROC to quit sending data. After which the EB is told to end, and then finally the ER is told to do the same. Each component, however, must wait for run control's end (or reset) event to come through before finally ending.

Similarly, the data flow through an EMU starts with the input channel. It goes through the modules in order and then through the output channel. When an EMU is shut down, the input channels/transporters must be the first to end, followed by the modules in order and finally the output channels/transporters. All parts must wait for the end (or reset) event to come through before finally ending.

7.3. *Triggered Data Format Channels*

There are 2 general formats for data flow. The first format is for data coming from a triggered DAQ system. The second is for data coming from a streaming, untriggered source. The channels described in this section are for a triggered DAQ.

7.3.1. ET Channels

When the ET system is used for data transport, it is usually done for performance purposes. The ET channel software is, therefore, multithreaded in such a way as to squeeze every last bit of speed out of it. The newly developed EtContainer class is used to provide garbage-free Java code which performs well under heavy load. Next is a brief description of both the input and output ET communication channels.

7.3.1.1. Output

Why not start with the most difficult part first? The ET output channel uses the interior class, DataOutputHelper, to do all the work.

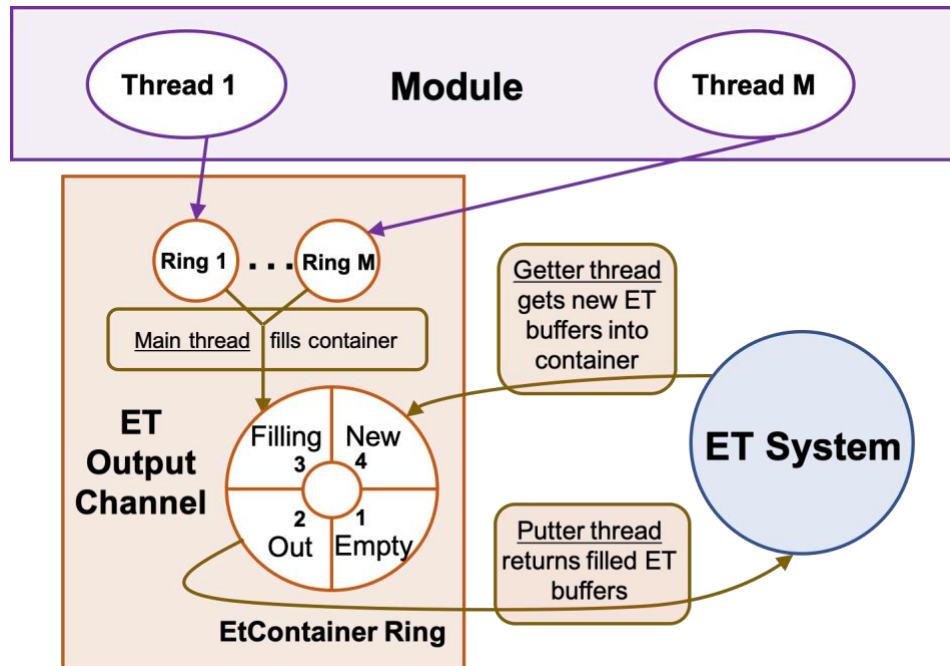


Figure 7.1 Et output channel internals

Figure 7.1 shows that it uses a fast ring buffer ([see the next chapter](#)) to do the coordination between 3 different threads. This ring has 4 slots, each of which is an EtContainer object containing “chunk” number of ET events (let’s call them buffers). Note that each ET buffer will eventually house data in evio format and will usually contain many evio events. I know, the double meaning of “event” is difficult.

- The “getter” thread gets “chunk” new buffers from ET and places them into one of the 4 container entries of the ring.
- The “main” thread then claims one of these containers with new events, from the ring. For each ET buffer in that container, the thread writes into it as many evio events from the module’s output as it can hold. Remember, the module has written evio events into this channel’s output rings (one output ring for each of the module’s event processing threads). So going from ring to ring, in proper sequence, this thread writes, in evio format, all the evio events from these output rings. When done filling all ET buffers with data, it marks the container as being ready to write. Note that for control and user events, each has its own ET buffer.
- Finally, there is “putter” thread which takes a full, ready-to-write container and places all those ET buffers back into the ET system.

If one of the evio events is the END event, all threads shut themselves down as it passes through. A reset command just stops all these threads.

7.3.1.2. Input

The ET input channel is a little simpler than the output. The input channel uses the interior class DataInputHelper to do all the work with 2 threads. Similar to the output, this channel has a ring of 2 EtContainers as seen in the next figure.

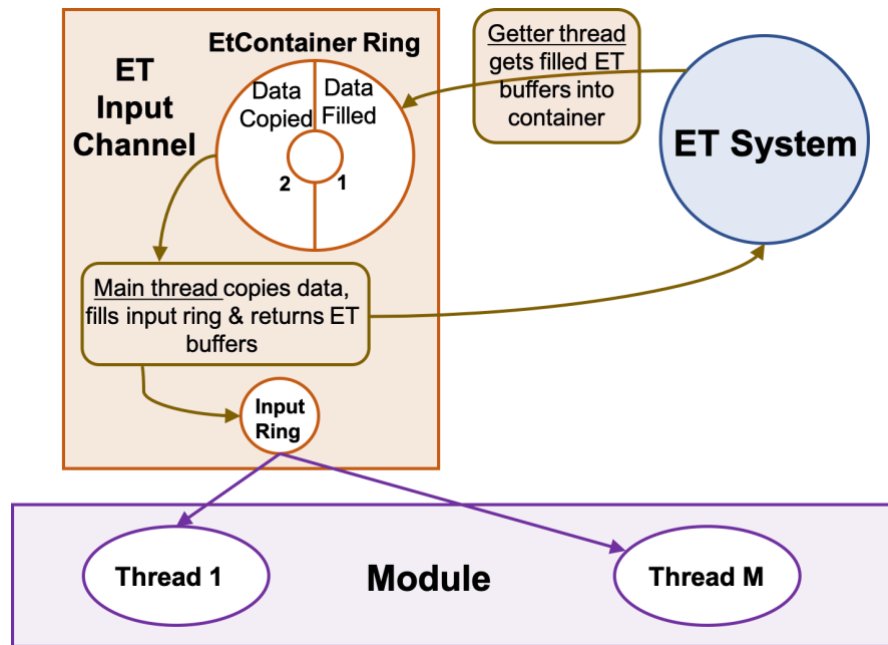


Figure 7.2 Et input channel internals

The getter thread gets an array of ET buffers in an EtContainer object to start with. The main thread then takes a newly filled container from the ring of 2, and for each ET buffer it copies the buffer, parses it, and places each resulting evio events into the channel's input ring. Finally, when all ET buffers are parsed, they're returned to the ET system and the container is marked for reuse. The input ring (not container ring) will be used as input to the first module. If an END event appears, it is placed on the ring, all ET events are returned and the thread exits.

Note that each ET buffer, when copied, is placed into a buffer provided from a ring-based supply of reusable buffers. Because each parsed evio event contains a reference to the buffer it was parsed from, the copy is necessary to prevent problems when the ET buffer is put back into the ET system and eventually reused – thereby changing the underlying buffer's data.

7.3.2. cMsg Channels, cMsg domain

7.3.2.1. Output

The output channel uses the interior class DataOutputHelper to do all the work with 1 thread to do the dispatching and a pool of threads (2 by default) to do all the writing. The number of writing threads can be set in the configuration by setting the wthreads attribute in the outchannel element. The dispatching thread grabs evio events from the module and stores them in ArrayLists (one for each writer) until either the count exceeds 1000 or the total memory exceeds 256KB in which case the lists are passed off to the write threads (control and user events get their own cMsg message). While the dispatching thread waits, the write threads will write the evio events into cMsg messages' byte arrays. When done, the dispatching thread will send the messages. This is not the fastest way to do things but that's not important if using cMsg to begin with.

7.3.2.2. Input

cMsg input is a little simpler than the output. The input channel has a subscription to a particular subject and type. The subject and type can be set as attributes in the configuration's inchannel element. If not explicitly set, the subject defaults to the name (attribute) of the inchannel and the type defaults to "data". Each time a message arrives, the subscription's callback is run and it parses the message's byte array and places the resulting banks on the channel's ring. The ring will be used as input to the first module.

If an END event appears, it is placed on the ring. The callback does nothing else, but the emu unsubscribes and then disconnects when the END or RESET commands reach the channel and the transport object.

7.3.3. cMsg Channels, emu domain

This channel was designed as a way to simplify the communication between the various DAQ components by using TCP sockets in a straightforward manner. But even the best of intentions go astray. The code implementing this is split between 2 libraries. The cMsg library contains the code to be an emu domain client – a process that connects to an emu domain server downstream and sends it data. On the other hand, the code implementing the server side is in the emu library since it was much easier to program it that way and it had access to the emu internals.

During initial testing, it became apparent that there was a bottleneck in the CPU processing power available to handle a single TCP socket which, in turn, limited the data rate between components. This was overcome by implementing multiple sockets to handle the data over 1 channel. This is configurable thru the “sockets” attribute in the config file and is settable in jcedit.

7.3.3.1. Output

The output is handled by a single DataOutputHelper thread that grabs evio events from the module placed in the output rings. Say there are 2 output TCP sockets. Each socket has its own associated ByteBufferSupply and sending thread. Each ByteBuffer supply is based on and acts as a ring buffer. The helper thread writes its evio events into a buffer from one of the sending threads and when full, it signals that sending thread to write the data over TCP. The next buffer written into will be from the next socket/sending-thread up in round robin fashion. Keeping things round-robin allows order to be preserved on the receiving end. Note that user or control events are always written into their own, exclusive buffer.

7.3.3.2. Input

The input is handled by (possibly) multiple DataInputHelper threads, each of which reads a single socket. Each input thread has an associated ByteBufferSupply which is also a ring buffer. A single input thread first reads a command, a size, then a buffer of data from the socket into a buffer from the supply. Based on the command, it will either continue parsing the buffer or realize it's an END event and exit.

Once the buffer is read into the supply, a single parser-merger thread will parse the buffer into evio events, merging everything from all the input threads. These evio events are placed into the channel's input ring from which the module will take data.

7.4. Streaming Data Format Channels

Recently the advent of streaming DAQ systems has become more prominent. The following channels were developed as a way to adapt CODA to this environment.

7.4.1. UDP

The `DataChannelImplUdpStream` class implements streaming channels over UDP. Although it is a general-purpose communication channel (functioning as either an input or output), it was developed mainly to handle data from hardware front-ends sent via UDP. The implementation and programming of UDP on an FPGA is much simpler than its TCP counterpart.

The trickiest thing to be aware of is the user-defined packet headers. Each packet sent has a header describing the contents of that packet and its place in a larger buffer. Its main purpose is to allow the packets to be reassembled by a receiver into complete buffers. The header sent by front-end hardware is of a particular format used by CODA.

In competition with that, the EJFAT project in combination with ERSAP have different formats. The ERSAP format is similar but different and plays the same role that the CODA header does. On top of that the EJFAT project inserts a second header at the very front of each packet in order to direct it through specialized network hardware called a "load balancer". Once through the hardware, that EJFAT header is stripped off.

Control of these headers is through the receiving component's configuration, set in `jcredit`. If the channel has:

```
useErsapHeader="on"
```

in its config file, then the ERSAP instead of the CODA header is used.

If an output channel has:

```
useLoadBalancer="on"
```

in its config file, then an EJFAT header is inserted at the front of each packet. In this case, the host specified in `jcredit` must be a load balancer NIC.

7.4.1. TCP

The `DataChannelImplTcpStream` class implements streaming channels over TCP in a very similar fashion to the emu domain channel with much of its code being borrowed from that channel. The main thing is that it handles a different data format.

7.5. Simulated ROCs and TS

In cases when it's necessary to test a DAQ system but actual data-producing ROCs are and trigger supervisors are unavailable, EMU-based ROCs and TS's can be used. In order

for such a simulation to work properly, all ROCs must have produced the same number of events when the run control commands to END or RESET are given, otherwise the EB will throw an exception. This is done by having ROCs sync with each other through the simulated Trigger Supervisor after every fixed number of events (20k by default) by sending cMsg messages - a poor man's trigger.

7.5.1. Trigger Supervisor

The TS is given a list of expected ROCs to sync in its configuration file. Using the platform's cMsg server (cMsg domain), it subscribes during prestart in the "RocSync" namespace to:

```
subject = "syncFromRoc",    type = "*"
```

This subscription's callback receives messages from the ROCs. From each message it gets the ROC's name from the "type" and whether that ROC has received the END command from run control or not from the "user int" (1 if END received, else 0). If all expected ROCs send such a message, then the callback sends a message back to all ROCs with:

```
subject = "sync",    type = "ROC"
```

If all ROCs have indicated that they have received the END command, then this message is sent with a user int of 1 meaning "stop sending events", else it's sent with a user int of 0 meaning "send the next batch of events". It then waits for another round of messages from the ROCs.

7.5.2. ROCs

In prestart, each simulated ROC connects to the platform's cMsg server and subscribes in the "RocSync" namespace to:

```
subject = "sync",    type = "ROC"
```

Once the GO command is received from run control, the ROC will start writing events. Once it has reached the predetermined limit (20k by default), it will stop and send a message to:

```
subject = "syncFromRoc",    type = "<roc name>"
```

where <roc name> is its actual name. If it has received the END command from run control, the message's "user int" is 1, else it's 0. Then it waits for an incoming message from the TS. The TS responds when all ROCs have finished their batch of events and sent their message. If that TS message contains a "user int" of 1, then the ROC quits producing events and ends, otherwise it writes another batch of events.

The END command actually gets blocked and waits until the writing thread gets the TS message to end things. At that point the writing thread allows the END command to proceed.

It is probable that each ROC receives the END command at a different point in its production of events. Yet this system allows all ROCs to end after having produced the exact same number of events. Say for example, that in a 2 ROC configuration, the first

ROC got an END after 295k events. It stopped after 300k, sent its message to the TS and waited for a response. Roc2, on the other hand, stopped at 300k but had not yet gotten an END. It also sent its message but indicated that it had no END and then waited for the TS response. The TS got the first ROC's message, but was waiting until the second also reported. When the TS had received both messages, one had not yet received an END so the TS told both to write another round of events. The second ROC eventually got an END but at event 310k. Thus when both stopped producing at 400k, the TS received their messages and realized that both had gotten the END and told them to end their event production. In this way all ROCs end up having produced the identical number of events and the EB will end nicely.

7.6. *Simulated Fixed-Rate ROCs and TS*

As in the previous section, there are cases when it's necessary or convenient to test a DAQ system with EMU-based ROCs and TSs. And having a ROC that produces data at a pre-determined fixed rate can be advantageous. Having such a ROC also requires a special TS designed to work with it.

The way the current fixed-rate TS is setup is to model Hall D which means setting a total desired data rate for 64 ROCS and adjusting the individual ROC's to match. This is easily changed by modifying the total desired data rate in the **TsFixedRateSimulation** constructor and by modifying the logic in the **callback()** method of the **InitCallback** interior class.

The very first thing that must be done is to hack the **Emu.java** file. Look into the **download(Command cmd)** method. The lines:

```
module = new RocSimulation(n.getNodeName(), attributeMap, this);
module = new TsSimulation(n.getNodeName(), attributeMap, this);
```

must be replaced with:

```
module = new RocFixedRateSimulation(n.getNodeName(), attributeMap, this);
module = new TsFixedRateSimulation(n.getNodeName(), attributeMap, this);
```

respectively.

In addition to the cMsg communication mentioned in the previous section, there are additional cMsg exchanges that need to take place. The tricky thing is that the fixed rate depends on the number of ROCs and how much data each sends. So there's a callback in the fixed-rate TS called **InitCallback** which can be modified to suit one's requirements. This callback needs to be executed before a run takes place.

To do that, each ROC contacts the fixed-rate TS in the GO transition sending an init message to the fixed-rate TS:

```
subject = "initFromRoc",    type = "<emu name>"
```

where the “user int” contains the number of bytes per entangled block of events including evio headers. Once sent, the ROC waits until the TS releases a latch. All ROCs get released at the same time and begin the normal sending of data.

Before releasing the ROCs, the TS’s callback figures out and sends in a message to all ROCs, the desired “entangled blocks/buffer” in the “user int” and the target “bufsPerSec” in an accompanying payload named “bufsPerSec” with:

```
subject = "init",    type = ROC
```

The ROC receives this info and passes it on to its output channels. Note that only the emu channel is capable of fixing its output rate by adding delays when necessary. The code for this is in the **DataChannelImplEmu.DataOutputHelper.flushEvent()** method.

Last, these classes are meant to be modified by the programmer seeking to simulate a particular experimental condition.

7.7. Evio Events Per ET Buffer

This feature is largely unused since the ET system is no longer used to handle the data transmission between the ROCs and the EBs.

Another largely hidden piece of controlling the data flow is instituted to avoid mismatches in the number of evio events coming from each ROC to the EB. ROCs with large amounts of data will send fewer evio events in an ET buffer than will those with little data. What we don’t want is for ROCs with only a little data to wait until its 2MB ET buffer is full before sending while in the meantime the EB has already received ET events from ROCs with big events. This keeps the EB waiting when it could be building.

To facilitate faster event building, all ROCs receive feedback from either the SEB or PEB (whichever is being used) in the form of cMsg messages, telling them the maximum number of evio events to fit into a single ET buffer before sending. The trick is to pick this number dynamically based on current DAQ conditions which will ensure good data flow.

Currently it works as follows. The SEB or PEB emu connects to the platform’s cMsg server in the cMsg domain in the namespace “M” and creates a subscription to:

```
subject = <emu name>,    type = "*"
```

The associated callback receives messages from the ET input channels containing the number of evio events in each ET buffer or M. This callback finds the lowest and highest M values from all reporting channels in order to send that number to the ROCs on the other end of the input channels. It also sends the highest safe value of M (a value that will allow the ROCs to operate without using the timeout to send data) which is calculated to be 2 x lowest-M. This feedback message is sent to the ROCs only if the highest safe value changes and no more than once every 2 seconds. Furthermore, once the feedback is sent, all M values return to their initial values and are recalculated. This keeps the feedback up to date within 2 seconds.

The ET input channels meanwhile report their current M value by publishing a cMsg message to:

```
subject = <emu name>,    type = "M"
```

with the user int set to M. They report no more than every ½ second in order to keep from overloading the cMsg server.

The feedback messages sent to the ROC are sent to the platform's cMsg server in the namespace <expid> with:

```
subject = <emu name>,    type = "eventsPerBuffer"
```

The highest safe M is sent as the user int while the low M and high M are sent as integer payload items called "lowM" and "highM" respectively.

Chapter 8

8 Fast Ring Buffers

Strictly speaking this chapter is really a part of the previous one about developer's details, but the fast ring buffers which are used throughout the emu deserve their own chapter. The folks at LMAX wrote a software package implementing fast ring buffers called the **Disruptor** which we'll be taking a close look at:

<https://github.com/LMAX-Exchange/disruptor>

A Jefferson Lab fork of it can be found at:

<https://github.com/JeffersonLab/disruptor>

For the moment, however, look at the [Figure 1.1](#) which shows queues between input channels and modules. A channel or module grabs an event off a queue, processes it, and places it on another queue. In the EB, for instance, there were 3 queues that each event spent time in. Originally, the queues used in the emu were of the type built into the Java language, like the `ArrayBlockingQueue`. Using this class was convenient and easy, but when the performance of the emu was profiled, 40% of the CPU time was taken up putting stuff on and taking stuff off of queues – not good. What is it about queues that are such a problem? Why are they so slow?

8.1. *Locks are Bad*

Let's look at an important component of queues – locks. Locks provide a thread-safe way to read and write common data, but are very expensive to use. When there is contention for a lock, the operating system must arbitrate resulting in context switching and suspended program threads waiting for the lock. While in control, the OS may decide to do other tasks during which the original context may lose cached data and instructions.

To demonstrate these effects the programmers at LMAX called a function which incremented a counter in a loop 500 million times under different conditions.

Method	Time (ms)
One thread	300
One thread with lock	10,000
Two threads with lock	224,000
One thread with CAS	5,700
Two threads with CAS	30,000
One thread with volatile write	4,700

Once locks are used, even uncontested, performance is 30x worse. With contention, it's 750x worse! Instead of using locks, atomic CAS or Compare-And-Swap operations can be used if the item to be updated is a single word. It's a much better approach since it does not require kernel arbitration and a context switch, but the processor must still lock its instruction pipeline to ensure atomicity and use a memory barrier to make the changes visible to other threads. The downside is that using only CAS operations and memory barriers to protect data beyond a simple counter is extremely difficult. Less costly in Java is the reading or writing of a volatile field which uses read and write memory barriers respectively.

8.2. *Cache Lines*

Let's look at one more subject of importance in queue performance – cache-lines. Hardware doesn't move memory around in bytes or words but in cache-lines that in linux are 64 bytes. What this means is that if two variables are in the same cache-line, and they are written to by different threads, they result in the same problem of write contention as if they were the same variable. This is known as “false sharing”. For minimal contention and therefore best performance, it is necessary that independent, but concurrently written, variables do not share the same cache-line.

8.3. *The Trouble with Queues*

The bounded queues originally used in the emu have write contention at the head, tail, and size variables. If there is more than one producer (the case for EB build threads writing to output channels), the tail pointer will be the point of contention as more than one thread wants to write to it. If there's more than one consumer (the case for EB build threads reading from input channels), then the head pointer is contended since this is not just a read but also a write as the pointer is updated when the element is consumed.

In actual use, queues are almost always full or empty since producers and consumers never run at the same rate. This translates into high levels of contention as the head and tail are the same pointer. And to top things off, the head, tail and size are often in the same cache-line resulting in false sharing. Okay, there's one more thing. In Java the queues are a constant source of garbage. Objects are created, stored on the queue,

removed and eventually discarded – nothing is recycled. If the queue is linked-list based, objects representing the nodes of the list need to be created and eventually reclaimed.

8.4. *Disruptor Design*

The LMAX disruptor is designed to address all the issues just mentioned. First off the memory usage is much better. All memory for the ring buffer is pre-allocated on startup. The ring is populated with an array of permanent objects or entries (therefore not garbage collected) that can contain data of interest. Allocating memory in this way allows traversal of the entries to be done in a very cache-friendly manner.

Since there is neither head nor tail, both consumers and producers track their own place, or sequence, as they go around the ring. Producers claim the next slot in sequence when claiming an entry in the ring. Since the emu only uses rings with one producer, this sequence of the next available slot is a simple, uncontested counter. Once a sequence value is claimed, its corresponding entry in the ring is now available to be written to by the claiming producer. When the producer is done with the entry, it can commit the changes by updating a separate counter which represents the cursor on the ring for the latest entry available to consumers. This it can do by using a memory barrier which is done by a volatile write in Java – no lock, no CAS. For better performance, the cursors are padded in such a way that false sharing never occurs.

Consumers, on the other hand, wait for a sequence to become available by using a volatile read of the cursor. Various strategies can be used to wait, but (probably) the fastest and the one used in the emu is a busy spin loop check of the cursor for 30,000 loops followed by a blocking strategy that uses a lock and condition variable. Consumers each contain their own sequence which they update as they process entries from the ring. These sequences allow the producer to track consumers in order to prevent the ring from wrapping, and they also allow other consumers to coordinate work on the same entry.

So we see that the disruptor ring buffers use no locks or CAS, minimize contention, and are cache-friendly. All concurrency is achieved using memory barriers.

An added benefit from this design is that when consumers are waiting for the next available sequence, the sequence actually read may be several entries beyond the next one. Thus only one cursor read is necessary for the processing of several entries. This type of batching increases throughput while reducing and smoothing latency at the same time.

8.5. *Disruptor Use in a Previous EB*

As an example of ring buffer use, consider the figure below which represents the ring buffer of one of the event builder's input channels (as it used to be since it's now been changed).

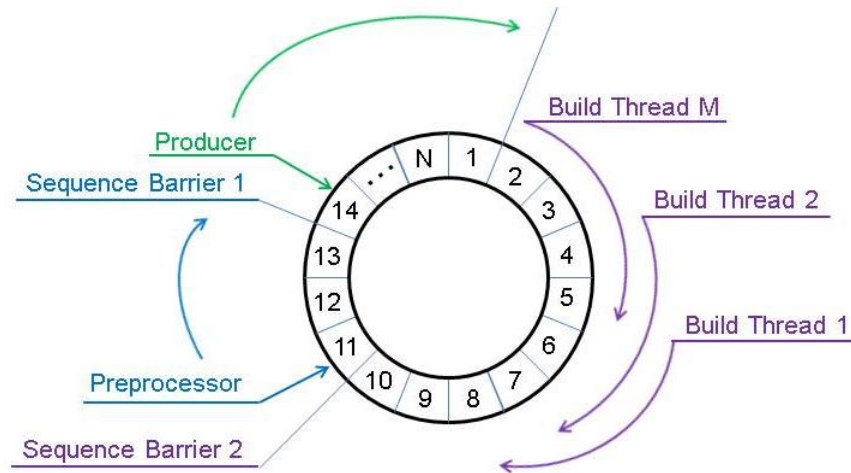


Figure 8.1 EB Input Channel Ring Buffer

In this case, the producer is an input channel. It starts with entry 1 and keeps placing evio events into the entries as data flows in and eventually ends up working on entry/event 14. When done with event 13, it updates its cursor saying that event 13 is available for consumers. That cursor is represented by “sequence barrier 1”.

The preprocessing thread is the first consumer. It reads the coda id of the sending coda component from the data and sees if it matches what is expected from the configuration (among other things). It starts consuming events as they become available and is currently working on event 11. When done with that it will read the value of sequence barrier 1 which is 13 and will be able to move to 12 and after that to 13 without reading the cursor again, and it will also update its sequence seen as “sequence barrier 2” in the figure. Just as the first barrier indicates when the producer is finished with an event, the second does that same thing to indicate that the preprocessor is finished with its event. All the build thread consumers are programmed to wait on the second barrier.

Each build thread takes one event from each input channel in order to build an event. Build thread 1 take event 1, build thread 2 takes event 2, etc. etc. Thus, in this case, build thread 1 will grab event 1, skip events 2 thru M, and go to event 1+M for its second and so on.

In the figure, the input channel cannot produce beyond event 1 (for the second time) since the build threads have yet to release any more.

Prior to using a single ring buffer, the input channel placed parsed evio events onto a queue which the preprocessing thread then used as its input. The preprocessor then placed its output on another queue which the build threads used as their input. All the build threads would contend for the second queue’s lock in order to read it. Thus 2 queues were replaced by only one ring buffer.

8.6. *Disruptor Use in the Byte Buffer Supply*

The **ByteBufferSupply** class (an instantiation of which is hence referred to as a **supply**) and its companion the **ByteBufferItem** class (an instantiation of which is hence referred to as an **item**) should really be in a separate library. They are 2 classes that work together to provide a generally useful, extremely low-latency, non-garbage producing source of reusable **ByteBuffer** objects.

The heart of what makes this work is a single disruptor ring containing **ByteBufferItem** objects. Such an object provides a wrapper of a **ByteBuffer** object. The supply can be used in 3 different modes:

- 1) It can be used as a simple supply of items. In this mode, only **get()** and **release()** are called. A user does a **get()**, uses that item/buffer, then calls **release()** when done with it. If there are multiple users of a single buffer (say 5), then call **item.setUsers(5)** before it is used and the buffer is only released when all 5 users have called **release()**.
- 2) As in the first usage, it can be used as a supply of items, but each item can be preset with a specific **ByteBuffer** object. Thus, it can act as a supply of buffers in which each contains specific data. Because of the circular nature of the ring used to implement this code, after all items have been gotten by the user for the first time, it starts back over with the first -- going round and round.

To implement this, use the constructor which takes a list of **ByteBuffer** objects with which to fill this supply. The user, instead of calling **get()**, calls **getAsIs()** which does **not** clear the buffer's position and limit. When finished reading/writing, user calls **release()**. It's up to the item's user to maintain proper values for the buffer's position and limit since it will be used again. If there are multiple users of a single buffer (say 5), then call **item.setUsers(5)** before it is used and the buffer is only released when all 5 users have called **release()**.

- 3) It can be used as a supply of items in which a single producer provides data for a single consumer which is waiting for that data. The producer does a **get()**, fills the buffer with data, and finally does a **publish()** to let the consumer know the data is ready. Simultaneously, a consumer does a **consumerGet()** to access the data once it is ready. The consumer then calls **release()** when finished which allows the producer to reuse the now unused buffer.

One general characteristic of a disruptor ring is that when a consumer of items releases an item, say item #100, all previous 99 items are released as well for any following consumers or the producer. In a supply, this is **not** desirable behavior. For example, if thread A is currently working with item 99 and thread B was simultaneously working with item 100 and has just released it, item 99 will unfortunately also be released, potentially allowing another user to grab it and use it at the same time – not good.

To mitigate this, **the supply is programmed to only release its items in sequence**. This, unfortunately, requires a lock. There is way around using a lock however. In one constructor, the user can guarantee that all items will be released in sequence, and

trusting the user, a lock is not used. Be warned, however, that releasing things out-of-order in that case will result in an exception or flaky behavior. In practice, the lock seems to have little effect on performance.

There is another condition in which a lock is used. If a user gets an item/buffer and wants it to be used in multiple (N) contexts/threads, the method `item.setUsers(N)` must be called. This item must be released in each of the multiple contexts in order to be given back to the supply in the final call to `release()`. The release of the item in each of the multiple contexts is locked if no initial guarantee of sequential item release is made. If such a guarantee was made in the constructor (as previously discussed), then a volatile variable is used to handle this instead of the less-preferable lock.

This feature of the supply was implemented to program the input channels of an EMU. In an ET or emu domain channel, there are buffers coming in either over the network or in shared memory. Each incoming buffer is copied into a buffer taken from a supply. This supply buffer is then parsed into multiple (N) evio events. These little evio event objects are what are placed into the channel's internal input ring buffer. Each of these N evio event objects are essentially pointers back into the original supply item/buffer. In the case of an event builder, each one of the multiple building thread grabs one of these evio events from each channel and builds it into a final event. When done building, each evio event is released by its build thread. When all N events are released, the buffer is released back to the supply. In the EB, built events themselves are also stored in a supply – one created for each build thread. Although one could create a new `ByteBuffer` for each built event, it would eventually have to be garbage collected resulting in a tremendous strain on the JVM.

The following is actual code used to create and use a supply.

```
// Create a supply of ByteBuffers
int ringSize = 1024; // number of buffers
int bufferSize = 256000; // bytes per buffer
ByteBufferSupply supply = new ByteBufferSupply(ringSize, bufferSize);

// Use the supply & double a single buffer size
ByteBufferItem item = supply.get();
item.ensureCapacity(2*bufferSize);
ByteBuffer buf = item.getBuffer();

// Release the buffer and allow for its reuse
supply.release(item);
```

Although the actual classes contain more complexity, they can be quite simple to use. The ring contains `ByteBufferItem` objects each of which contain a single `ByteBuffer`. In this way, the contained `ByteBuffer` objects can be replaced easily if more memory is needed.

8.7. General Disruptor Use in the Emu

Ring buffers are used in several places throughout the emu:

- Each input channel has one ring
- Each output channel has one ring for each of the preceding module's event-processing threads
- Internal operation of the ET output channel
- The ByteBufferSupply, used to provide reusable ByteBuffers, has one ring and is used by, for example,
 - EB build threads
 - Emu input channel
 - ET input channel
 - Simulated ROC

All input channels place their parsed evio events into 1 ring buffer. All output channels have the same number of ring buffers as the last module has event-processing threads (build threads in the EB). Thus, each event-processing thread has its own ring buffer in each output channel – one producer per ring. This design eliminates multiple producers and contested writes. One can see this clearly in [figure 7.1](#).

8.8. *Ring Buffer Example Code*

The code to create a ring is:

```
// Number of ring entries must be power of 2
int ringSize = 1024;

// Define factory to produce objects contained in the ring
final private class RingItemFactory implements EventFactory<RingItem> {
    final public RingItem newInstance() {
        return new RingItem();
    }
}

// Create the ring buffer using a very fast busy-spin waiting strategy
RingBuffer<RingItem> rb = createSingleProducer(new RingItemFactory(),
        ringSize, new YieldingWaitStrategy);
```

The code to produce data for a ring is:

```
while(true) {
    // Get the next available sequence or ring slot (may block)
    long sequence = rb.next();

    // Get the actual entry at that sequence
    RingItem entry = rb.get(sequence);

    // Fill entry with data ...

    // Release the entry back to the ring buffer for consumers
    rb.publish(sequence);
}
```

A note on publishing the sequence, it not only releases its corresponding entry to all consumers that follow, but it also releases all previous entries. For example, if a producer does several “rb.next()” and “rb.get()” calls without publishing, publishing the last sequence will also release all prior ones.

The code to consume data from a ring is:

```
// Object allowing us to wait for producer to be finished
SequenceBarrier barrier = rb.newBarrier();
// Object allowing us to tell others where we are
Sequence sequence = new Sequence(Sequencer.INITIAL_CURSOR_VALUE);
// If this is the last consumer before the producer,
// in other words, no other consumers are depending on us going first,
// then add the following line.
rb.addGatingSequence(sequence);
// Keep track of our place in the ring
long nextSequence = sequence.get() + 1;

while(true) {
    // Get the next available sequence or ring slot (may block).
    // May be > the sequence asked for (nextSequence).
    long availableSequence = barrier.waitFor(nextSequence);

    // While we have access to available entries ...
    while(nextSequence <= availableSequence) {
        // Get the actual entry at the next sequence
        RingItem entry = rb.get(nextSequence);

        // Read entry data ...

        // Go to the next ring entry
        nextSequence++;
    }
    // Release all used entries back to the ring
    // buffer for producer or dependent consumers
    sequence.set(availableSequence);
}
```

The code is rather straight forward. Although not shown here, additional barriers can be added so that the ring understands that certain consumers depend on other consumers going first.

Appendix A

A. CODA Types

The following is a list of CODA DAQ component types, their default run control priority levels and their descriptions.

CODA Type	Default Priority	Description
TS	1210 (master ROC)	Trigger Supervisor
GT	1110	
FPGA	960	Streaming ROC
ROC	1010	Readout Controller
DC	910	Data Concentrator (first level event builder)
EBER	810	Combined EB & ER, used with DCs or ROCs
PEBER	810	Combined EB and ER, used with ROCs
SEBER	810	Combined EB and ER, used with DCs
SAGG	660	Secondary Time Slice Aggregator
SEB	610	Secondary Event Builder (2nd level event builder used with DCs)
PAGG	560	Primary Time Slice Aggregator
PEB	510	Primary Event Builder (one and only one event builder)
FCS	410	Farm Controller
ER	310	Event Recorder
SLC	110	Slow Control Component
USR	10	User Component
EMU	0	Event Management Unit